

Amazon Bedrock AgentCore & Strands SDK

Deep Technical Research Report — Architecture, Security, Operations, and Roadmap (April-June 2026)

Prepared for Enterprise Platform Architects

July 2026

Contents

1	Executive Summary	5
2	Part I — Platform Foundations	6
2.1	1. Why AgentCore Exists	6
2.2	2. The AgentCore Service Map	6
3	Part II — Runtime	8
3.1	3. Runtime Architecture	8
3.1.1	3.1 Isolation Model	8
3.1.2	3.2 Session Lifecycle and Stickiness	8
3.1.3	3.3 Cold Starts and Latency Engineering	8
3.1.4	3.4 Protocols, Versioning, and Endpoints	8
3.1.5	3.5 Networking	9
3.1.6	3.6 Pricing Model	9
4	Part III — Gateway	10
4.1	4. Gateway Architecture	10
4.1.1	4.1 What Gateway Actually Is	10
4.1.2	4.2 The Dual-Sided Security Architecture	10
4.1.3	4.3 Request Lifecycle	11
4.1.4	4.4 Semantic Tool Search	11
4.1.5	4.5 Ingress Networking	11
5	Part IV — Policy	12
5.1	5. Policy Engine Deep Dive	12
5.1.1	5.1 Why Policy Exists	12
5.1.2	5.2 Cedar: Why This Language	12
5.1.3	5.3 Evaluation Semantics	12
5.1.4	5.4 Neuro-Symbolic Policy Authoring	12
5.1.5	5.5 Rollout Discipline: LOG_ONLY → ENFORCE	13
5.1.6	5.6 Policy + Lambda Interceptors: Composability, Not Competition	13
5.1.7	5.7 Relationship to IAM	13
5.1.8	5.8 GA Timeline and Guardrails Integration	14
6	Part V — Registry	15
6.1	6. Registry Architecture (Preview)	15
6.1.1	6.1 Purpose and Positioning	15
6.1.2	6.2 The Four-Persona IAM Model	15

6.1.3	6.3 What Registry Deliberately Does <i>Not</i> Do (Preview Limitations)	15
6.1.4	6.4 Registry vs. Gateway — the Control-Plane / Data-Plane Split	15
7	Part VI — Harness	17
7.1	7. Harness Architecture	17
7.1.1	7.1 What Harness Is, and Isn't	17
7.1.2	7.2 What GA Auto-Provisions	17
7.1.3	7.3 Multi-Model, Mid-Session Provider Switching	17
7.1.4	7.4 Built-In Tools and the Inline-Function Escape Hatch	17
7.1.5	7.5 Positioning Against Runtime and Third-Party “Managed Agent” Offerings	18
7.1.6	7.6 Pipeline and CI/CD Integration	18
8	Part VII — Identity	19
8.1	8. Identity Architecture	19
8.1.1	8.1 Core Model: Delegation, Not Impersonation	19
8.1.2	8.2 Components	19
8.1.3	8.3 Inbound vs. Outbound Auth	19
8.1.4	8.4 Workload Identity as a Trust Boundary (Confused-Deputy Prevention)	19
8.1.5	8.5 Federation	20
8.1.6	8.6 Identity Propagation Across Multi-Agent Chains	20
9	Part VIII — Memory	21
9.1	9. Memory Architecture	21
9.1.1	9.1 Two-Tier Model	21
9.1.2	9.2 Built-In Long-Term Strategies	21
9.1.3	9.3 Extraction Pipeline	21
9.1.4	9.4 Governance, TTL, and Multi-Tenant Isolation	21
10	Part IX — Browser and Code Interpreter	23
10.1	10. Browser Tool	23
10.2	10. Code Interpreter	23
10.2.1	10.2.111.1 Design Intent	23
10.2.2	10.2.211.2 Independent Security Findings — A Material Correction to the Isolation Claim	23
11	Part X — MCP Server Hosting	25
11.1	11.112. Hosting MCP Servers on AgentCore Runtime	25
12	Part XI — Strands Agents SDK Deep Dive	26
12.1	12.113. Architecture and Philosophy	26
12.2	12.214. Tools	26
12.3	12.315. Model Abstraction	26
12.4	12.416. MCP and A2A as First-Class Tool Sources	26
12.5	12.517. Multi-Agent Orchestration Patterns	27
12.6	12.618. Hooks, Guardrails, and the Harness-SDK Extension	27
12.7	12.719. Observability Built In	27
12.8	12.820. Deployment Models	27
12.9	12.921. Documented Production Gaps (Independent Assessment)	27
12.10	12.102. Strands ↔ AgentCore Service Mapping	28
13	Part XII — Observability	29
13.1	13.123. AWS-Native: AgentCore Observability + CloudWatch GenAI Observability	29
13.2	13.224. Phoenix (Arize)	29
13.3	13.325. Comparative Recommendation: CloudWatch-Only vs. Phoenix-Only vs. Hybrid	29
13.4	13.426. Distributed Tracing, Span Hierarchy, and Root-Cause Analysis	30

14Part XIII — Security: Complete Threat Model	31
14.127. Threat Model Overview	31
14.1.127.1 Prompt Injection	31
14.1.227.2 Tool (Injection) Poisoning / MCP Poisoning	31
14.1.327.3 Data Exfiltration	32
14.1.427.4 Cross-Agent Attacks (Multi-Agent Trust Boundaries)	32
14.1.527.5 Privilege Escalation / Confused Deputy	32
14.1.627.6 Replay Attacks and Token Theft	32
14.1.727.7 Secret Management	33
14.1.827.8 Runtime, Browser, and Code Interpreter Isolation — Summary Assessment	33
15Part XIV — Production Architecture	34
15.128. Network Connectivity Patterns	34
15.229. Multi-Account and Multi-Region Patterns	34
15.330. Disaster Recovery: Active-Active vs. Active-Passive	34
15.431. Hybrid Cloud	35
16Part XV — Release Analysis: April-June 2026	36
16.132. Chronological Timeline	36
16.233. Feature-Level Deep Dives	37
16.2.133.1 Policy GA — Migration Guidance	37
16.2.233.2 Harness GA — Migration Guidance	37
16.2.333.3 Registry Preview — Adoption Guidance	37
16.2.433.4 Payments Preview — Risk Framing	37
17Part XVI — Roadmap Prediction	39
17.134. Method	39
17.235. Next 6 Months (through ~December 2026)	39
17.336. Next 12 Months (through ~mid-2027)	40
17.437. Next 24 Months (through ~mid-2028)	42
18Part XVII — Adjacent Roadmap Signal: AWS Continuum and AWS Context	45
19Part XVIII — Best Practices	46
19.138. Runtime and Harness	46
19.239. Gateway and Policy	46
19.340. Identity	46
19.441. Memory	46
19.542. Code Interpreter and Browser	47
19.643. Observability	47
20Part XIX — Anti-Patterns	47
21Part XX — Production Readiness Checklist	48
22Part XXI — Cost Optimization Guide	48
23Part XXII — Security Hardening Guide (Summary)	49
24Part XXIII — Keeping the Agent Live: Resilience, Rollout, and Control Operability	50
24.144. Stress Testing and Load Testing	50
24.245. Kill Switch	50
24.346. Circuit Breakers	51
24.447. Canary Release and Progressive Rollout	52

24.547a. Feature Gates	53
24.648. Resume Workflow	53
24.749. Failover and Timeout/Retry Discipline	54
24.850. Exception Handling at the Agent, MCP, and Tool Layer: Auth vs. Business Exceptions	55
24.951. Human-in-the-Loop Switch	56
24.102. Sampling	57
24.153. Synthesis: The Full “Keep It Live” Stack	58
25Appendix A — Sources	59
26Appendix B — Glossary	59

1 Executive Summary

Amazon Bedrock AgentCore is AWS’s managed platform for taking AI agents from prototype to production. It is deliberately **not** an agent framework — it is a set of composable, independently-adoptable services (Runtime, Gateway, Identity, Memory, Policy, Registry, Harness, Browser, Code Interpreter, Observability, Evaluations, Optimization, and the newly previewed Payments) that sit underneath any agent framework, including AWS’s own open-source **Strands Agents SDK**, LangGraph, CrewAI, LlamaIndex, the OpenAI Agents SDK, and the Claude Agent SDK. The platform’s thesis, repeated consistently across AWS messaging since its October 2025 preview launch, is that the hard part of agentic AI was never writing the agent loop — it is securing tool calls, isolating sessions, managing identity across trust boundaries, and observing non-deterministic behavior at scale.

Between April and June 2026, AgentCore moved through its most consequential quarter to date:

- **Policy** (Cedar-based, deterministic, gateway-enforced authorization) reached general availability on **March 3, 2026**, and by June had grown Lambda-interceptor composability, Bedrock Guardrails integration, and natural-language-to-Cedar authoring.
- **AgentCore harness**, a fully declarative “two API calls to a production agent” abstraction built on Strands, went from preview (April 22, 2026) to general availability (**June 18, 2026**, announced at the AWS New York Summit), with multi-model mid-session switching, managed memory by default, and Step Functions integration.
- **Registry**, a governance catalog for agents, tools, MCP servers, and “skills,” entered public preview (~April 2026) with a four-persona (Admin/Publisher/Approver/Consumer) IAM model but no auto-indexing, no federation, and no IaC support yet.
- **Payments** entered preview (May 7, 2026), letting agents autonomously transact against x402-priced APIs and MCP servers via Coinbase and Stripe wallet connections.
- **Gateway** added GA support for fronting AgentCore Runtime agents directly as targets, expanded OAuth 3LO/OBO token-exchange patterns, and gained Lambda request/response interceptors that compose with Cedar policy evaluation.
- **Observability** deepened its native CloudWatch GenAI Observability integration (Evaluations, Optimization/failure-insights) while remaining fully OpenTelemetry-compatible with Arize Phoenix, Datadog, LangSmith, Langfuse, and Braintrust.
- Independent security research (Palo Alto Unit 42, BeyondTrust Phantom Labs, Sonrai Security) found and AWS partially remediated real isolation gaps in Code Interpreter’s “Sandbox” network mode — a material finding for any threat model built on AgentCore’s isolation claims.
- At the same June 17 summit, AWS previewed two **adjacent but non-AgentCore** platform services — **AWS Continuum** (autonomous vulnerability remediation with staged learn-mode/enforce-mode trust) and **AWS Context** (an identity-aware enterprise knowledge graph) — that signal where the next 12–24 months of “trustable autonomy” investment is headed.

This report reverse-engineers each AgentCore service down to its request flow and trust boundary, maps the Strands SDK’s internals onto that platform, builds a threat model grounded in both AWS’s stated design and independent red-team findings, and closes with evidence-scored predictions for the platform’s roadmap through mid-2028. Throughout, findings are graded by source strength: **AWS-documented** (official docs/blogs/release notes), **AWS-stated-marketing** (keynotes, press claims not yet independently verified), and **independent research** (security researchers, practitioner blogs, benchmarks).

2 Part I — Platform Foundations

2.1 1. Why AgentCore Exists

AWS’s own framing, repeated by GM Madhu Parthasarathy and VP David Richardson across 2025–2026 talks, is that most agent pilots built with open-source frameworks stalled before production because teams had to hand-roll session isolation, credential handling, tool governance, and tracing — the same undifferentiated heavy lifting AWS solved once for compute (EC2), storage (S3), and containers (Fargate/Lambda). AgentCore is explicitly framework- and model-agnostic: it will run a LangGraph agent, a Strands agent, or hand-written Python with equal support, and it will call Bedrock-hosted Claude, Nova, or externally-hosted OpenAI and Gemini models. This is a deliberate strategic choice — AWS does not need to win the framework war to win the infrastructure layer beneath it, and by remaining agnostic it captures workloads regardless of which framework wins developer mindshare.

Strands Agents SDK, AWS’s own open-source (Apache 2.0) agent framework, occupies a special position: it is the framework AWS itself uses to power AgentCore harness internally, but it is not required to use AgentCore, and AgentCore is not required to use Strands. This report treats them as a paired reference implementation because that pairing is how AWS’s own samples, tutorials, and the harness are built.

2.2 2. The AgentCore Service Map

As of June 2026, “AgentCore” refers to the following distinct, independently billed and independently adoptable services:

Service	Status (June 2026)	GA Date / Milestone
Runtime	GA	GA since ~late 2025; quota increases through June 2026
Gateway	GA	GA since 2025; Runtime-as-target GA in Q2 2026
Identity	GA	GA since 2025
Memory	GA	GA 2025; Episodic strategy added Dec 2025
Code Interpreter	GA	GA since 2025
Browser	GA	GA since 2025
Observability	GA	GA since 2025; CloudWatch GenAI Observability tie-in Dec 2025
Policy	GA	March 3, 2026
Evaluations	GA	March 31, 2026
Optimization (recommendations, failure insights)	GA	~May–June 2026
Harness	GA	June 18, 2026 (preview since April 22, 2026)
Registry	Preview	~April 2026, five regions
Payments	Preview	May 7, 2026 , four regions
MCP Server hosting (via Runtime)	GA	Part of Runtime since 2025

A critical architectural distinction threads through the whole platform: **Gateway is the data**

plane (it proxies and enforces policy on live traffic), while **Registry is the control plane** (it catalogs and governs resources at build/publish time). Confusing the two — for example, assuming Registry auto-indexes what Gateway serves — is a common early-adopter mistake documented by multiple practitioners in the April 2026 wave of Registry-preview write-ups.

3 Part II — Runtime

3.1 3. Runtime Architecture

3.1.1 3.1 Isolation Model

AgentCore Runtime’s foundational design decision is **one Firecracker microVM per session**, not per request and not a shared multi-tenant container pool. Firecracker is the same open-source VMM AWS uses for Lambda and Fargate; AWS states microVMs boot in under 125ms, which is what makes per-session hardware isolation economically viable at agent scale. Each microVM gets its own isolated CPU, memory, and filesystem. When a session ends, the entire microVM is terminated and its memory is sanitized — there is no reuse of session state across users, and a compromised agent process cannot observe or interfere with another session’s memory, tool state, or execution context by design.

This is a stronger isolation boundary than the shared-container model most teams reach for by default (a single Fargate service or Lambda execution environment reused across many users’ conversations), and it is the architectural reason AWS positions Runtime as suitable for regulated, multi-tenant workloads without additional per-tenant infrastructure.

3.1.2 3.2 Session Lifecycle and Stickiness

A session is identified by a client-supplied or Runtime-generated `runtimeSessionId`. Subsequent calls using the same ID are routed (“stuck”) to the same microVM via a session header, preserving in-memory state, environment variables, running processes, and filesystem content without the agent needing to reload context. This stickiness is **best-effort, not a correctness guarantee** — the microVM will be recycled on `maxLifetime` or `idleRuntimeSessionTimeout`, and AWS documentation and `re:Post` guidance are explicit that MCP servers hosted on Runtime must be built **stateless between individual HTTP requests**, externalizing any state that must survive a recycle event to AgentCore Memory, DynamoDB, Redis/Valkey, or S3. Relying on process-local RAM as anything other than a latency optimization is a documented anti-pattern.

3.1.3 3.3 Cold Starts and Latency Engineering

An AWS `re:Post` deep-dive (Angelino, Arzhanov, Gaafar, Moeller — May 2026) decomposes Runtime startup latency and recommends: container image optimization, strategic pre-warming pings, multiple endpoints for blue/green traffic shaping, and self-maintained warm pools. The warm-pool pattern is economically sound specifically because of AgentCore’s consumption-based pricing — memory is billed separately and more cheaply than active compute time, so holding idle warm microVMs open is materially cheaper than the business cost of cold-start latency for latency-sensitive, high-traffic agents. Teams should monitor real traffic and right-size the warm pool per time-of-day rather than over-provisioning statically.

3.1.4 3.4 Protocols, Versioning, and Endpoints

Runtime supports three inbound protocols: plain HTTP (REST request/response), MCP (Streamable HTTP, JSON-RPC, both stateless and stateful modes), and A2A (Agent-to-Agent). Every configuration change (container image, protocol, network settings) creates a new **immutable version**; a **DEFAULT endpoint** auto-updates to the newest version, while named endpoints (`dev`, `staging`, `prod`) can be pinned and repointed independently — giving zero-downtime rollback by simply repointing an endpoint alias, the same pattern as Lambda aliases/versions.

3.1.5 3.5 Networking

By default Runtime deploys in **PUBLIC** network mode. **VPC mode** provisions AWS-managed ENIs inside customer-specified subnets (via the `AWSServiceRoleForBedrockAgentCoreNetwork` service-linked role), enabling private access to RDS, internal APIs, and on-prem systems via Direct Connect/VPN, without traversing the public internet. A May 2026 AWS Networking blog documents four progressively more locked-down patterns: (1) public endpoint, public egress; (2) VPC egress via ENI, public ingress; (3) PrivateLink ingress + VPC egress, blocking public ingress via a resource-based policy condition on `aws:SourceVpce`; (4) full isolation — no IGW/NAT at all, with every AWS service call routed through VPC endpoints. Pattern 4 is the correct target state for regulated workloads handling PII, PHI, or financial data.

3.1.6 3.6 Pricing Model

Runtime is consumption-based: compute is billed only while active (not during LLM inference I/O wait, though session state is held), and separate, cheaper memory-holding charges apply for idle/warm microVMs. Direct code deployment incurs standard S3 storage costs; container deployment incurs standard ECR costs. A frequently cited industry estimate (Cloudvisor, May 2026) puts AgentCore infrastructure cost at roughly 10–30% of total agent cost at scale, with model inference (billed separately through standard Bedrock pricing) dominating the bill for most workloads — a moderate-traffic support agent (10k conversations/month) was estimated at \$50–200/month infrastructure plus \$200–800/month inference.

4 Part III — Gateway

4.1 4. Gateway Architecture

4.1.1 4.1 What Gateway Actually Is

AgentCore Gateway is a fully-managed, protocol-translating **AI gateway** — not merely an “MCP proxy.” It is the single, secure entry point through which agents reach three categories of downstream target:

- **MCP targets** (Lambda functions, OpenAPI specs, Smithy models, existing MCP servers, built-in connector templates such as Salesforce/Slack/Jira/Asana/Zendesk) are **aggregated** — Gateway acts as one virtual MCP server that merges the tools/list of every attached target into a single unified catalog, exposed to the client as one consolidated tools/list response.
- **HTTP targets** (other agents, A2A services, or an AgentCore Runtime agent added directly as a target) are **proxied directly**, without aggregation or protocol translation.
- **Inference targets** route LLM traffic across multiple model providers through one unified, model-based routing endpoint.

Gateway differs from a conventional **API Gateway** in that it natively speaks and translates *agent* protocols (MCP, A2A) rather than just REST/HTTP, and it differs from a generic **service mesh** in that its core function is protocol *aggregation and translation* into a single MCP surface plus **dual-sided OAuth enforcement**, not east-west traffic shaping between microservices. It differs from a bare **MCP proxy** in that it adds Cedar policy enforcement, Lambda interceptors, semantic tool search, and unified observability — a raw MCP proxy has none of these.

4.1.2 4.2 The Dual-Sided Security Architecture

Gateway enforces authentication and authorization on **both sides** of every call:

Inbound (client → Gateway). Gateway acts as an OAuth **resource server**, validating tokens against a configured identity provider (Cognito, Okta, Auth0, Entra ID, or a custom OIDC provider). Supported inbound modes: OAuth (JWT) for token-based authorization, IAM (AWS SigV4) for AWS-identity-based authorization, `AUTHENTICATE_ONLY` (validate the token but delegate authorization to the target — required for token-passthrough patterns), or no authorization (dev/test only).

Outbound (Gateway → target). Five supported patterns: 1. **No authorization** — explicitly discouraged. 2. **IAM-based (SigV4)** — the gateway service role signs requests; AWS can auto-provision this role via the console/CLI and auto-attaches least-privilege permissions per target added. 3. **Caller IAM credentials** — Gateway assumes a role on behalf of the caller via the Federated Access Service (FAS), preserving the caller’s identity in the signed request. 4. **OAuth 2LO (client credentials)** — pure machine-to-machine, no user in the loop. 5. **OAuth 3LO (authorization code grant)** — user-delegated access; requires the Gateway to have been created with MCP protocol version 2025-11-25 or later, and requires a registered `returnUrl` for the post-consent redirect. 6. **Token exchange grant (On-Behalf-Of / OBO)** — Gateway exchanges the inbound user’s access token for a new, downstream-scoped token that carries **both** the user’s and the agent’s identity, so downstream services can enforce fine-grained authorization at every hop without re-prompting for consent. 7. **Token passthrough** — the inbound token is forwarded unmodified (requires `AUTHENTICATE_ONLY` inbound mode); the target validates it itself. 8. **API key** — AgentCore-generated keys for simple target auth.

Credentials for every outbound pattern are brokered and cached through **AgentCore Identity’s Token Vault** — Gateway itself never persists long-lived secrets.

4.1.3 4.3 Request Lifecycle

1. Client sends an MCP `tools/call` (or `tools/list`) request to the Gateway's single endpoint, bearing an inbound credential (JWT or SigV4).
2. Gateway validates the inbound credential against the configured authorizer.
3. If a **Lambda REQUEST interceptor** is attached, it runs next — this is where request enrichment happens: injecting tenant IDs, exchanging a bearer token for tenant-scoped credentials, adding geography/role context that downstream Cedar policy will evaluate.
4. If a **Policy engine** is attached, Cedar evaluates the (possibly-enriched) request: principal, action (mapped from the specific tool name), resource (the gateway ARN), and context (arbitrary input fields, including anything the interceptor injected). Default is deny; a matching forbid always wins over a matching permit.
5. On ALLOW, Gateway translates the MCP call into the target's native protocol — a Lambda Invoke, an OpenAPI HTTP call, a Smithy-modeled call, or a passthrough to another MCP server/agent — using the resolved outbound credential.
6. The target executes and returns a result.
7. If a **Lambda RESPONSE interceptor** is attached, it can filter the tool list dynamically or redact sensitive fields in the response before it reaches the client.
8. Every hop — auth decision, policy decision, interceptor invocation, target latency — streams to AgentCore Observability / CloudWatch.

4.1.4 4.4 Semantic Tool Search

As the aggregated tool catalog behind a Gateway grows into the hundreds, stuffing every tool schema into the model's context window becomes wasteful and degrades tool-selection accuracy. Gateway ships a special built-in tool, `x_amz_bedrock_agentcore_search`, callable through the standard MCP `tools/call` operation, that performs semantic retrieval over the tool catalog so an agent can discover only the handful of tools relevant to its current task rather than being handed the entire catalog on every turn.

4.1.5 4.5 Ingress Networking

Gateway supports AWS PrivateLink interface endpoints for fully private ingress (`com.amazonaws.<region>.bedrock-agentcore.gateway`), letting resources inside a customer VPC reach Gateway without traversing the public internet. Note the documented caveat: identity-provider round-trips for OAuth (token retrieval, consent redirects) and Gateway's own outbound calls to external MCP targets still require internet egress — PrivateLink secures the *ingress* path from caller to Gateway, not every downstream hop.

5 Part IV — Policy

5.1 5. Policy Engine Deep Dive

5.1.1 5.1 Why Policy Exists

AWS’s own security-blog framing (May 2026) is direct about the problem: an LLM’s plan cannot be trusted to enforce its own constraints. System prompts and training-time alignment are bypassable via prompt injection or hallucination; hard-coded checks scattered through tool code are unauditable at scale. Policy’s answer is to move authorization **entirely outside the agent and outside the LLM’s context**, into the Gateway boundary, so that a compromised or manipulated agent cannot reason its way around a rule it cannot see or influence. Policy is enforced regardless of *how* the agent was prompted, jailbroken, or buggy — the enforcement point is structurally separate from the reasoning loop.

5.1.2 5.2 Cedar: Why This Language

Policy is built on **Cedar**, AWS’s open-source (and, as of early 2026, CNCF-hosted) authorization policy language — the same language behind Amazon Verified Permissions. Cedar was chosen for three properties: it is human-readable, it is machine-analyzable via automated/formal reasoning, and policies are schema-validated at authoring time. AgentCore auto-generates the Cedar schema from each Gateway’s actual tool definitions, which is the key differentiator from generic Verified Permissions (where the developer must hand-author the schema and call authorization APIs themselves). Policy in AgentCore is purpose-built for exactly one enforcement point — the Gateway’s MCP request path — with agent-specific features (partial-evaluation tool filtering, natural-language authoring) built in.

5.1.3 5.3 Evaluation Semantics

- **Default deny.** If no policy matches a request, the result is DENY.
- **Forbid overrides permit.** If any forbid policy matches, the result is DENY even if a permit also matches.
- **At least one permit required.** ALLOW requires at least one matching permit and zero matching forbid policies.
- Evaluation happens **per tool call** (tools/call), distinct from tools/list — a caller can be permitted to *see* a tool exists without being permitted to *invoke* it, and vice versa in some designs.

A minimal example (paraphrased from AWS’s getting-started guide) permits a caller tagged with a specific username to invoke a refund tool only when the requested amount is below a threshold — the amount check happens against `context.input.amount`, a field the Gateway injects automatically from the tool call’s actual arguments at evaluation time.

5.1.4 5.4 Neuro-Symbolic Policy Authoring

Administrators can write Cedar directly, or describe a rule in plain English (e.g., “allow callers with manager scope to use `markdown_to_email`”) and have it formalized automatically. AWS’s security blog describes this as a **neuro-symbolic feedback loop**: an LLM proposes candidate Cedar from the natural-language description, and **Cedar Analysis** — a symbolic, mathematical reasoning engine — validates the candidate against the gateway’s schema and checks the *entire policy set* holistically for conflicts, redundancy, overly-permissive grants, and overly-restrictive grants, failing the operation with a description of the problem if issues are found. This control-plane analysis runs every time a policy is attached, not just at natural-language authoring time.

Practitioner guidance (multiple independent write-ups, May 2026) converges on treating natural-language generation as a **draft**, not a final artifact: always inspect the generated Cedar against the real Gateway schema before enforcing, and prefer specific, concrete natural-language requirements (“allow refund:write callers to process refunds when amount is less than 500”) over vague ones (“make refunds safe”), since vague requirements produce vague — and often over-broad — Cedar.

5.1.5 5.5 Rollout Discipline: LOG_ONLY → ENFORCE

Every policy engine attaches to a Gateway in one of two modes:

- **LOG_ONLY** — every request is evaluated and the decision is logged to CloudWatch, but nothing is blocked. This is the recommended starting mode: it lets a team validate that real production traffic maps to the policy set the way they expect before any enforcement risk is taken.
- **ENFORCE** — decisions are actually applied; unpermitted calls are rejected (commonly surfaced as an MCP-level error / HTTP 403), and every decision is still logged for audit.

The consistent practitioner rollout pattern: start LOG_ONLY, inspect what *would* have been denied, fix tool-name/claim-mapping/schema mismatches, then flip to ENFORCE only once the denied-case set is fully understood and expected.

5.1.6 5.6 Policy + Lambda Interceptors: Composability, Not Competition

An important architectural nuance from AWS’s June 2026 blog on combining the two mechanisms: **REQUEST interceptors always run before Cedar policy evaluation**. This ordering is intentional — interceptors are the right tool for anything *dynamic* (external lookups, token exchange, injecting per-tenant context that isn’t in the raw request), while Cedar is the right tool for anything expressible as a *static logical rule* over the resulting enriched context. AWS documents three composable design patterns:

1. **Policy-only** — role-based tool restriction via straightforward permit/forbid rules. Best when authorization is a pure function of identity claims already present in the token.
2. **Interceptor-only** — dynamic credential exchange (e.g., swapping a JWT for tenant-scoped IAM credentials, an “act-on-behalf” pattern) with no static rule needed.
3. **Combined** — an interceptor enriches the request with attributes an external system must supply (e.g., the caller’s data-residency region from a lookup service), and Cedar enforces the resulting rule (e.g., “EU-tagged callers may not invoke US-only tools”). RESPONSE interceptors additionally can filter which tools appear in tools/list per-caller and redact sensitive fields from tool results — dynamic response shaping that Cedar’s request-time evaluation cannot do.

The design guidance AWS gives: use interceptors for everything inherently dynamic and Cedar for everything expressible as a logical condition; treat them as a pipeline, not overlapping controls.

5.1.7 5.7 Relationship to IAM

IAM and Policy are complementary layers answering different questions. IAM answers “is this AWS principal (the Gateway’s execution role) allowed to invoke this Lambda function at all?” Policy answers “is this specific end-user, acting through this agent, allowed to call process_refund with amount=2000 right now?” Removing IAM scoping under the assumption that “Policy handles authorization now” removes an entire independent layer of defense-in-depth — the two must both be correctly configured, not treated as substitutes.

5.1.8 5.8 GA Timeline and Guardrails Integration

Policy reached GA on **March 3, 2026**. By the April-June wave, AWS integrated **Bedrock Guardrails directly into the Policy/Gateway layer**: Guardrails now evaluates outputs from *already-authorized* agent actions and inputs to gateway targets for prompt-injection attempts, harmful content, and sensitive-data exposure — critically, this evaluation runs at the Gateway layer, **outside the agent’s own context window**, so the agent cannot reason around a check it never sees. Because every tool and every context source is required to route through the Gateway, AWS’s framing is that *every new agent capability is automatically governed by the same security layer* without additional integration work per tool. AWS has also signaled (as “coming soon” in the same announcement) that third-party detection signals — Check Point, Zscaler, Rubrik, Netskope, and SentinelOne were named — will be pluggable into the same policy evaluation pipeline.

6 Part V — Registry

6.1 6. Registry Architecture (Preview)

6.1.1 6.1 Purpose and Positioning

AgentCore Registry is a fully managed, serverless **catalog and governance layer** — the control-plane counterpart to Gateway’s data-plane role. It provides centralized discovery, versioning, approval workflows, and search across four resource categories: **Agents, MCP Tools, Skills, and Custom Resources** (arbitrary JSON for anything that doesn’t fit the other three). It entered public preview around April 2026 in five regions (us-east-1, us-west-2, ap-southeast-2, ap-northeast-1, eu-west-1), free during preview, with usage-based “Net Records” pricing planned at GA.

6.1.2 6.2 The Four-Persona IAM Model

Registry is architecturally unusual within AgentCore for explicitly separating four distinct personas into distinct IAM policies:

- **Administrator** — owns registry infrastructure: creates/configures registries, sets authentication mode (IAM or JWT), wires EventBridge for approval automation, and decides whether auto-approval is enabled (documented as *always off* by default in production-oriented guidance).
- **Publisher** — submits new records for approval.
- **Approver** — reviews and approves/rejects submissions (a distinct persona from Administrator, enabling segregation of duties).
- **Consumer** — discovers and consumes registry records (search, retrieve) with no write/mutate permissions; practitioner guidance recommends running Consumer-side MCP proxies with a `--read-only` flag to structurally prevent accidental writes.

6.1.3 6.3 What Registry Deliberately Does *Not* Do (Preview Limitations)

Multiple independent practitioner write-ups (April 2026) converge on the same gap list, worth treating as a due-diligence checklist before depending on Registry in a production governance program:

- **No auto-indexing.** Deploying an agent to Runtime does not automatically publish it to Registry — registration is a manual, explicit act.
- **No cross-account/cross-team federation.** Each AWS account has its own independent registries; there is no native mechanism to expose a platform-team’s registry to squad-level accounts.
- **No SemVer-aware version diffing.** Registry supports a `recordVersion` field but does not compute or enforce semantic-version compatibility between versions.
- **No Terraform or CloudFormation support as of April 2026** — provisioning is Console/CLI/boto3/API only, which several teams noted as a blocker for full infrastructure-as-code workflows.
- The **Registry ID** is not surfaced as a labeled, copyable field in the console — it must be extracted from the resource ARN, a minor but repeatedly-noted UX friction point.

6.1.4 6.4 Registry vs. Gateway — the Control-Plane / Data-Plane Split

The clearest mental model documented across multiple AWS-partner blogs: Gateway is the **traffic layer** — it proxies and enforces policy on live invocations. Registry is the **build-time layer** — it catalogs what exists, who owns it, and whether it has been vetted, but it does not sit in the request path at runtime. A common integration pattern emerging in mid-2026

write-ups is using Registry purely as a **metadata catalog for “skills”** (name, description, instructions, allowed-tools list) that an agent loads dynamically per-session — keeping the actual tool implementations in-process for latency, while decoupling skill *content* (editable without a redeploy) from the agent *runtime* (which only needs to change when the loading mechanism itself changes). One documented migration (a 16-skill AWS governance agent) reported moving from hardcoded, always-loaded skill descriptions in the system prompt to Registry-backed dynamic loading, cutting prompt bloat and enabling live skill edits without a container rebuild — completed end-to-end, per that team’s account, in about an hour using AI-assisted planning.

7 Part VI — Harness

7.1 7. Harness Architecture

7.1.1 7.1 What Harness Is, and Isn't

AgentCore harness is a **fully managed agent orchestration abstraction built on top of Runtime**, internally powered by the Strands Agents framework. Where Runtime asks a developer to write the agent loop and ship a container, Harness asks only for **configuration**: a model, a system prompt, and a set of tools, via two API calls — `CreateHarness` (define) and `InvokeHarness` (run). AWS's own framing at GA: "two API calls to a production-grade agent." No orchestration code, no Dockerfile, no ECR push is required, though an execution role, Bedrock model access, and (for cross-model use) IAM permissions on the harness/runtime action families are still needed.

Harness entered **public preview on April 22, 2026** and reached **GA on June 18, 2026** at the AWS New York Summit.

7.1.2 7.2 What GA Auto-Provisions

At GA, omitting a memory ARN on `CreateHarness` auto-provisions a managed AgentCore Memory resource with sensible defaults: **SEMANTIC + SUMMARIZATION** strategies, 30-day short-term event expiry, AWS-owned encryption, and multi-tenant namespace isolation keyed on `actorId` by default. GA also added: the AWS-curated skills catalog behind a one-toggle setup; built-in evaluations and A/B testing with statistical-significance reporting; unified observability auto-traced to CloudWatch; immutable versioning with named endpoints and instant rollback (the same version/endpoint model as Runtime); and **export to Strands code** when configuration alone is no longer sufficient and full control is needed (export to the Claude Agent SDK was announced as "coming soon" at GA).

7.1.3 7.3 Multi-Model, Mid-Session Provider Switching

A default model is set at `CreateHarness` time; any individual `InvokeHarness` call can override it to one of four provider families — `bedrock` (any Bedrock-hosted model: Claude, Nova, Llama, DeepSeek, Qwen, Kimi, MiniMax, Cohere, Mistral, and GPT-5.5/GPT-5.4 via Bedrock Mantle), `openAi` (direct OpenAI API), `gemini` (direct Google Gemini), or `liteLlm` (any LiteLLM-supported provider). The documented and independently-verified behavior that distinguishes this from a naive "swap the model ID" implementation: **conversation history and message context carry across the swap**. A session can plan its approach with one model, execute a step with a second, and summarize with a third, without losing continuity — independent verification (a June 2026 hands-on write-up) confirmed this by swapping Claude Sonnet 4.6 for `gpt-oss-120b` mid-conversation and observing that context (including a specific detail seeded earlier in the session) survived the swap.

7.1.4 7.4 Built-In Tools and the Inline-Function Escape Hatch

Every Harness session ships two always-available built-in tools — `shell` and `file_operations` — running inside an isolated environment with its own filesystem, giving the agent a working directory it can read from and write to safely. Beyond that, Harness supports five configurable tool types. The most architecturally significant is the **inline function**: it lets Harness pause execution mid-turn (streaming a `stopReason: "tool_use"` event back to the caller), hand control to the caller's own application code, and resume once the caller returns a tool result. Because the inline function's code runs in the **caller's** environment — not inside Harness's managed compute — this is the sanctioned pattern for giving an agent access to

resources behind a corporate firewall (internal databases, on-prem APIs) without exposing them to AgentCore’s compute layer at all.

7.1.5 7.5 Positioning Against Runtime and Third-Party “Managed Agent” Offerings

A widely-cited comparison (“Latent Thoughts,” April 2026) frames the choice as: pick **Harness** when the fastest path to a working agent matters more than custom control (Harness *inverts* the deployment model — you deploy configuration, not code); pick **Runtime** directly when you need full control over the orchestration loop, custom middleware, or a framework Harness doesn’t wrap; and treat “Bedrock Managed Agents, powered by OpenAI” (a separate, OpenAI-specific offering combining OpenAI’s own harness with AWS infrastructure) as a narrower alternative scoped to teams committed to OpenAI’s frontier models specifically. One functional gap noted as of the April 2026 preview period: unlike Runtime, Harness was at that time reachable only via the AWS API/SDK/CLI (InvokeHarness) — it did not yet expose an HTTP, MCP, or A2A endpoint the way Runtime does; whether this expands is an open roadmap question addressed in Part IX.

7.1.6 7.6 Pipeline and CI/CD Integration

Harness integrates directly into **AWS Step Functions** via an InvokeHarness state, letting a harness-defined agent participate as a step inside a larger orchestrated pipeline without custom Lambda glue code.

8 Part VII — Identity

8.1 8. Identity Architecture

8.1.1 8.1 Core Model: Delegation, Not Impersonation

AgentCore Identity’s foundational design principle is that an **agent authenticates as itself while carrying verifiable user context** — it does not impersonate the end user. This is what AWS’s identity-chaining documentation calls separating *identity* from *delegation* and then combining them only within defined boundaries. The chain, end to end:

1. A human user authenticates through an existing IdP (Cognito, Okta, Entra ID, Auth0) and the client application receives a user JWT.
2. The client invokes the agent, passing the user JWT as inbound auth.
3. AgentCore Runtime/Gateway validates the JWT and extracts user identity context.
4. AgentCore Identity issues a **Workload Access Token** — an internally AWS-signed token representing *both* the agent’s own workload identity and the user context, without simply forwarding the raw user token.
5. When the agent needs a downstream, non-AWS resource (Google Drive, Slack, GitHub, Salesforce), it presents the Workload Access Token to the **Token Vault**, which either returns a cached, previously-consented provider token or orchestrates a fresh OAuth consent flow (3LO), then caches the result bound to the specific agent+user pair.
6. The agent uses the vaulted, scoped provider credential to call the external resource — never the user’s raw session credential, and never a shared system-wide credential.

Every hop preserves both identities and produces an audit trail; if the user later revokes consent, the Token Vault blocks further use of that provider credential immediately.

8.1.2 8.2 Components

- **Agent Identity Directory** — a unified directory of agent/workload identities, each with an ARN, metadata (name, OAuth return URLs, timestamps), managed centrally. Agents are first-class security principals, not applications masquerading as users.
- **Agent Authorizer** — validates whether a caller (user or service) may invoke a given agent at all (the inbound check).
- **Resource Credential Provider** — stores the configuration needed to obtain credentials for a specific downstream resource server.
- **Resource Token Vault** — encrypted at rest and in transit, stores OAuth access/refresh tokens, client credentials, and API keys; the single source of truth an agent queries at invocation time rather than embedding secrets in code.

8.1.3 8.3 Inbound vs. Outbound Auth

Inbound Auth governs who may invoke a Runtime, Gateway, or tool — configured as IAM (SigV4) or JWT (OAuth/OIDC). A single Runtime supports *either* SigV4 *or* JWT inbound auth, not both simultaneously; teams needing both must create separate Runtime versions/endpoints. **Outbound Auth** governs how the agent/Gateway reaches downstream, non-AWS resources — via an API key or an OAuth client (2LO or 3LO), referenced by ARN in agent/tool code.

8.1.4 8.4 Workload Identity as a Trust Boundary (Confused-Deputy Prevention)

Each deployed Runtime automatically provisions a corresponding workload identity directory keyed on the Runtime’s own ARN. This directory enforces a **callback-URL whitelist** for

OAuth redirects — only URLs explicitly registered against that specific workload can complete an authorization-code flow on its behalf, preventing a malicious or misconfigured client from hijacking the redirect to exfiltrate an authorization code. Separately, AWS’s guidance is explicit and repeated across multiple official and community sources: execution-role trust policies **must** include `aws:SourceArn/aws:SourceAccount` conditions scoped to the specific Gateway or Runtime ARN. Without this condition, *any* AgentCore Runtime in *any* AWS account could in principle assume the role via the shared `bedrock-agentcore.amazonaws.com` service principal — a textbook confused-deputy vulnerability that AWS’s own samples explicitly guard against.

8.1.5 8.5 Federation

Identity natively integrates with Cognito, Okta, and Microsoft Entra ID as inbound identity providers, avoiding a re-platforming of existing enterprise identity. A practical May 2026 guide (Arpan Das) documents the “configure once, enforce everywhere” pattern: a single Entra App Registration and a single AgentCore OAuth Client entry govern both MCP-server (tool) invocations and full Agent Runtime invocations, differentiated only by which OAuth 2.0 grant type applies to a given caller pattern (a human via an MCP client, a pipeline invoking an autonomous agent, or a chained multi-agent scenario needing identity propagation).

Cross-cloud limitation, independently documented (Descope, June 2026): AgentCore Identity, like its Azure and GCP counterparts, is tightly coupled to its own cloud. A non-AWS agent can register against AgentCore, but still ultimately needs AWS credentials to participate — and there is no AWS-native mechanism to unify agent identity across AWS, Azure AI Foundry, and Vertex AI simultaneously. Third-party “agentic identity hub” vendors (Descope is cited as one example) are positioning specifically to fill this cross-cloud identity-federation gap, which is a legitimate architectural gap for genuinely multi-cloud agent estates rather than a vendor-created problem.

8.1.6 8.6 Identity Propagation Across Multi-Agent Chains

When a supervisor agent delegates to a specialist agent, the user’s identity claims travel with the delegated request. Combined with Gateway-enforced Cedar policy, this means a monitoring/specialist agent several hops downstream can still only access the original user’s data — the cryptographic binding between workload identity and user identity holds regardless of how many agents sit in the chain, which is the property that makes multi-agent architectures auditable rather than an identity-laundering risk.

9 Part VIII — Memory

9.1 9. Memory Architecture

9.1.1 9.1 Two-Tier Model

AgentCore Memory splits into **short-term memory** (the raw, turn-by-turn conversation buffer for a single session — every message, unprocessed) and **long-term memory** (persistent, cross-session, semantically searchable knowledge, produced asynchronously by one or more configurable **strategies**). This distinction carries an important timing contract that is easy to get wrong: long-term memory is populated by an *asynchronous* extraction pipeline that runs after short-term events are recorded, not synchronously mid-session — a strategy is not available to influence the *current* turn the way short-term context is.

9.1.2 9.2 Built-In Long-Term Strategies

AgentCore ships four built-in strategies, each with a distinct retrieval contract:

- **Semantic** — extracts discrete facts and knowledge; retrieval is pure relevance-ranked semantic search over embeddings. Best for “what does the agent know about X.”
- **User Preference** — captures stable behavioral signals (“prefers window seats,” “budget-conscious”). Only processes USER/ASSISTANT-role messages.
- **Summary** — produces XML-structured, <topic>-tagged condensed narratives of a session; unlike Semantic and User Preference, Summary processes *all* message roles, capturing tool-call context too.
- **Episodic** (added Q4 2025, expanded through 2026) — captures not just facts but *how the agent arrived at them*: goal, reasoning steps, actions taken, outcome, and a reflection. This is the strategy AWS’s VP of AgentCore, David Richardson, described publicly as designed to answer questions like a returning traveler’s preferred seat or price range without needing custom instructions — the agent recalls relevant episodes automatically when retrieval conditions are met, rather than requiring the developer to hand-code “remember this.”

Teams can further customize any built-in strategy’s extraction/consolidation system prompt or swap its underlying model while keeping the output schema fixed, or build a fully **self-managed strategy** with custom extraction, consolidation, and ingestion logic (billed differently from built-in strategies).

9.1.3 9.3 Extraction Pipeline

When new short-term events are recorded, an asynchronous pipeline invokes an LLM to identify meaningful information matching the configured strategy or strategies, producing structured memory records in a predefined schema. Strategies run in **parallel** — multiple memory types process independently and do not block one another, which keeps the pipeline’s latency profile flat as more strategies are added. Retrieval (`RetrieveMemoryRecords`) performs semantic search via embeddings automatically; unlike some competing memory frameworks that expose explicit recency/importance/relevance weighting knobs, AgentCore’s approach treats relevance as embedding-driven and handles recency/importance implicitly inside consolidation.

9.1.4 9.4 Governance, TTL, and Multi-Tenant Isolation

Short-term event expiry is configurable per Memory resource (a 7-day default is common in samples; Harness’s GA-provisioned default is 30 days). Namespace templates key long-term memory to an `actorId`, giving each user/tenant a structurally isolated memory space — Alice’s

preferences are never retrievable in Bob's session, even across entirely different session IDs, because retrieval is scoped by namespace, not merely filtered post-hoc.

10 Part IX — Browser and Code Interpreter

10.1 10. Browser Tool

AgentCore Browser is a serverless, cloud-hosted browser automation service purpose-built for agent-driven web interaction (form filling, data extraction, QA testing, CAPTCHA-heavy flows). Architecturally it shares Runtime’s session model: each browser session is isolated (VM-level sandboxing), auto-scales without infrastructure management, and supports VPC connectivity for reaching internal web applications privately. Observability is first-class — AWS documents live session viewing, CloudTrail logging, and full session replay for compliance review and troubleshooting, positioning Browser as auditable by design rather than a black box a human must trust blindly.

10.2 11. Code Interpreter

10.2.1 11.1 Design Intent

Code Interpreter gives an agent a secure, isolated sandbox for executing agent-generated code — critical because arbitrary code execution driven by an LLM is one of the highest-consequence capabilities an agent can be granted. AWS’s architecture: each session runs in its own isolated sandbox (backed, like Runtime, by ephemeral hardened microVMs), with pre-built multi-language runtimes, large-file support (up to 100MB inline, 5GB via S3), and CloudTrail logging. Default execution time is 15 minutes, extendable to 8 hours for long-running data-processing workloads. Sandboxes support session-level customization (compute resources, available libraries) and two network modes: **Sandbox** (advertised as complete isolation, no external access) and **Public** (internet-connected).

10.2.2 11.2 Independent Security Findings — A Material Correction to the Isolation Claim

This is one of the most consequential findings in this report, because it directly bears on any threat model that assumes Code Interpreter’s “Sandbox” mode provides *complete* network isolation.

- **Palo Alto Networks Unit 42 (April 30, 2026)** demonstrated that Sandbox mode, originally documented by AWS as providing “complete isolation with no external access,” in fact permitted DNS resolution — an operative gap, since DNS queries are themselves a viable, low-bandwidth data-exfiltration channel (DNS tunneling). AWS updated its documentation to clarify that Sandbox mode allows DNS resolution by design, rather than treating it purely as a bug.
- **BeyondTrust Phantom Labs (originally published March 16, 2026, updated April 22, 2026)** independently disclosed the same underlying gap — Sandbox mode’s DNS resolution enabling DNS-based command-and-control/exfiltration — and reported that, following public disclosure, **AWS did subsequently remediate the DNS-exfiltration vector specifically** (the update notes DNS-based exfiltration is “no longer possible” as of the April 22 update). BeyondTrust’s practical guidance for teams that had assumed Sandbox mode meant complete isolation: inventory all Code Interpreter instances and their network modes; scan for prompt-injection vectors that could manipulate code sent to the interpreter; apply Guardrails on inputs as a second layer; and — the clearest structural fix — **migrate genuinely sensitive workloads to VPC-only mode**, which AWS states is the only mode providing complete network isolation and full control over DNS resolution.
- **Sonrai Security (March 25, 2026)** took the finding a step further: even in Sandbox mode, the interpreter’s IAM role credentials remained reachable and abusable via access to a subset of AWS services (notably S3) that did not require external network access

to reach — meaning “no external network access” did not, in Sonrai’s testing, equate to “no ability to abuse the role’s AWS permissions.” Sonrai’s conclusion: network-mode isolation and IAM-permission scoping are two independent controls, and a team that hardens one while assuming it also hardens the other has a false sense of security.

Threat-model implication: “Sandbox mode” should not, by itself, be treated as a sufficient boundary for code execution against untrusted or adversarially-influenced input (e.g., code whose content or targets were partly shaped by a prompt-injected instruction). The two concrete architectural mitigations independent researchers converge on are (1) **VPC-only network mode** for any Code Interpreter session handling sensitive workloads, and (2) **least-privilege, tightly-scoped IAM execution roles** for the interpreter specifically — never reuse a broadly-privileged role across the interpreter and other AgentCore primitives.

11 Part X — MCP Server Hosting

11.1 12. Hosting MCP Servers on AgentCore Runtime

AgentCore Runtime can host a developer’s own MCP server as a managed, scaling endpoint — distinct from Gateway, which *aggregates and translates* existing APIs/Lambdas into MCP tools. When a Runtime is configured for the MCP protocol, the platform expects the container to expose `0.0.0.0:8000/mcp`, matching the default path most official MCP SDKs use out of the box. Runtime supports both **stateless** (`stateless_http=True`, the recommended default for basic tool servers) and **stateful** (`stateless_http=False`, required for elicitation, LLM-generated sampling, or progress notifications) streamable-HTTP modes; in either mode, Runtime auto-injects an `Mcp-Session-Id` header on any request lacking one, so clients can maintain continuity to the same session.

The critical operational caveat, independently confirmed by multiple practitioners and AWS’s own re:Post guidance: AgentCore does **not** guarantee sticky routing of every individual HTTP request to one specific container instance under horizontal scale-out — session affinity via `runtimeSessionId` is a best-effort optimization, not a correctness mechanism. Any MCP server design that depends on in-process RAM for protocol-level session state will break unpredictably once traffic triggers scaling or a microVM recycle. The only correct architecture externalizes state — AgentCore Memory, DynamoDB, Redis/Valkey, or S3 are the documented options — and treats the `Mcp-Session-Id/runtimeSessionId` purely as a routing hint, never as a durability guarantee.

Both IAM (SigV4) and OAuth/Cognito authentication work out of the box for Runtime-hosted MCP servers, and the same Gateway-fronting pattern available for HTTP agents (adding a Runtime as a Gateway HTTP target) applies to MCP-protocol Runtimes as well — letting a team put Cedar policy, Guardrails, and unified observability in front of a hand-written MCP server without changing the server’s own code.

12 Part XI — Strands Agents SDK Deep Dive

12.1 13. Architecture and Philosophy

Strands Agents is AWS’s open-source (Apache 2.0), model-driven agent SDK, available for Python and TypeScript, and the framework that internally powers AgentCore harness. Its central design bet is the **agentic loop over an orchestration graph**: rather than requiring a developer to hand-wire a state machine of steps, Strands hands the model the conversation, the system prompt, and descriptions of available tools, and lets the model itself decide — each turn — whether to respond in natural language, plan a sequence of steps, reflect on prior steps, or invoke one or more tools. This is explicitly the **ReAct** (Reasoning and Acting) pattern; Strands’ contribution is a clean, minimal implementation of that loop plus production plumbing (streaming, hooks, multi-agent orchestration, MCP, observability) around it.

```
Agent(tools=[...], model=..., system_prompt=...)
  → invoke(prompt)
    → loop:
      1. Call model with [system_prompt, history, tool_specs]
      2. Model responds with text and/or tool_use blocks
      3. If tool_use: execute tool(s), append ToolResult to history
      4. Repeat until model produces a final text response
    → return AgentResult (text + full trace + metrics)
```

12.2 14. Tools

Any Python function decorated with `@tool` (or, in TypeScript, defined via the `tool()` helper with a Zod input schema) becomes callable by the model — no separate registration step, no adapter class. During local development, Strands supports **hot-reloading**: point it at a directory and newly added or edited tool files are picked up automatically without restarting the agent process. This is explicitly flagged by independent reviewers (Starlog, May 2026) as a **production security liability if left enabled** — hot-reload implies arbitrary code execution on file modification with no sandboxing, versioning, or access control, and the framework does not itself warn against this in-band. The correct operational posture is to treat hot-reload as strictly a local/dev-only feature and ensure it is disabled (or simply absent, by not mounting a watched directory) in any Runtime/Harness container deployed to production.

A community-maintained `strands-agents-tools` package supplies 20+ pre-built tools (calculator, HTTP requests, AWS API wrappers, a semantic-search retrieve tool over Bedrock Knowledge Bases, and more).

12.3 15. Model Abstraction

Strands normalizes message formats, streaming protocols, and tool-calling conventions behind a single `Model` interface, with first-class providers for **Amazon Bedrock** (default), **Anthropic** (direct API), **OpenAI**, **Google Gemini**, **Ollama**, and **LiteLLM** (which in turn opens dozens of additional providers). Community-contributed providers extend this further (Cohere, xAI, Fireworks AI, NVIDIA NIM, vLLM, MLX, SGLang). Swapping providers is a one-line change (`BedrockModel(...)` → `OpenAIModel(...)`), which is precisely the abstraction Harness exposes at the API level via its four provider-family override.

12.4 16. MCP and A2A as First-Class Tool Sources

Strands treats an `McpClient` as a `ToolProvider` — it can be passed directly into an Agent’s `tools=[...]` list alongside plain Python functions, with the SDK handling connection lifecycle (Python requires a `with` context manager; TypeScript manages this implicitly). Multiple

transports are supported (stdio, Streamable HTTP, SSE); for AWS-hosted MCP servers using SigV4 authentication, the community `mcp-proxy-for-aws` package handles AWS credential brokering transparently. Strands also implements the **Agent-to-Agent (A2A)** protocol, letting agents call other agents as tools — this is the SDK-level primitive that AgentCore Gateway’s HTTP-target passthrough mode is designed to route at the platform level.

12.5 17. Multi-Agent Orchestration Patterns

Strands ships three built-in multi-agent topologies:

- **Graph** — nodes are agents, edges define explicit hand-off order; deterministic, developer-defined routing (e.g., researcher → writer).
- **Swarm** — agents decide their own routing at runtime; each agent chooses whether to hand off to a peer or produce the final response, making the execution path model-driven rather than developer-fixed.
- **Workflow** — natural-language-defined multi-step task decomposition for consistency across complex, repeatable processes (via the separate agent-sop companion project).

12.6 18. Hooks, Guardrails, and the Harness-SDK Extension

Strands exposes lifecycle hooks (`BeforeToolCallEvent` and others) that let a developer intercept, log, validate, or veto any step of the loop — for example, canceling a tool call whose arguments fail a business rule before it ever executes, entirely inside the agent process. A separate, newer package, **harness-sdk** (the open-source counterpart to the managed AgentCore harness), extends this pattern with **Steering handlers** that let an agent correct itself mid-course rather than failing silently, and traces every decision by default.

12.7 19. Observability Built In

Every `Agent.invoke()` call returns an `AgentResult` carrying full trace and metrics data by default — no separate instrumentation step is required to get *basic* visibility into what the agent did. For production-grade distributed tracing, Strands emits standard **OpenTelemetry (OTEL)** spans, which is the exact mechanism that makes it trivially compatible with AgentCore Observability, Arize Phoenix, Datadog, LangSmith, Langfuse, W&B Weave, and Braintrust simultaneously — the SDK does not pick a proprietary telemetry format.

12.8 20. Deployment Models

Strands agents are “just Python” (or Node) — they can run anywhere: locally, on Lambda, Fargate, EC2, EKS, or wrapped via the `BedrockAgentCoreApp` helper and deployed directly to AgentCore Runtime via the AgentCore CLI or container workflows. AWS’s reference architectures document separating concerns between the agentic loop (running in one environment) and tool execution (running in an isolated backend, e.g., Lambda-hosted tools called from a Fargate-hosted loop), as well as a **return-of-control** pattern where the *client* — not the agent’s own runtime — is responsible for actually executing a requested tool call, which is the SDK-level analog of Harness’s inline-function pattern described in Part VI.

12.9 21. Documented Production Gaps (Independent Assessment)

An independent, critical review (Starlog, May 2026) — while broadly positive on Strands’ developer experience — flags concrete production gaps a platform team should plan around rather than assume are solved by the SDK alone:

- **No built-in session-management abstraction** beyond the conversation buffer — no native vector store for semantic memory, no conversation-summarization utility. (AgentCore Memory is the documented AWS-native answer to this specific gap when running on AgentCore.)
- **Primitive error handling** — a failing tool call’s exception bubbles up and terminates the agent by default; there is no automatic retry, no circuit breaker, and no structured mechanism to hand a failure back to the model so it can try an alternative approach. Production deployments need custom error boundaries wrapped around tool invocations.
- **MCP subprocess execution and hot-reloading are both flagged as security concerns for multi-tenant or untrusted-input contexts** — neither ships with sandboxing, resource limits, or access controls by default; this is a framework-level gap that AgentCore’s platform-level isolation (microVMs, Code Interpreter sandboxing) is specifically designed to compensate for when Strands runs *on* AgentCore, but the gap is real if Strands is run bare, off-platform.

12.10 22. Strands ↔ AgentCore Service Mapping

Strands SDK Concept	AgentCore Service It Maps To
Agent loop execution	Runtime (or Harness, which wraps Strands automatically)
@tool functions, custom Python tools	Executed in-process inside the Runtime microVM, or offloaded to Gateway-fronted Lambda/API targets
McpClient tool sources	Gateway (aggregated MCP targets) or direct Runtime-hosted MCP servers
Model provider abstraction	Harness’s multi-provider InvokeHarness override, or direct Bedrock/Anthropic/OpenAI/Gemini calls from Strands code
Session/conversation state	AgentCore Memory (short-term); externalized state stores for MCP protocol state
Hooks (BeforeToolCallEvent, etc.)	Complements, but does not replace, Gateway Lambda interceptors and Cedar Policy — hooks run <i>inside</i> the trusted agent process; Policy/interceptors run <i>outside</i> it, which is the security-relevant distinction
OTEL trace emission	AgentCore Observability / CloudWatch GenAI Observability, or any OTEL-compatible backend (Phoenix, Datadog, etc.)
A2A protocol	Gateway HTTP-target passthrough for agent-to-agent traffic
Local dev / hot-reload	Explicitly not a production pattern; disable before deploying to Runtime/Harness
harness-sdk (open-source)	Conceptual sibling to the managed AgentCore harness — same steering/guardrail philosophy, self-hosted

13 Part XII — Observability

13.1 23. AWS-Native: AgentCore Observability + CloudWatch GenAI Observability

AgentCore Observability is available uniformly across every AgentCore service (Runtime, Gateway, Policy, Memory, Browser, Code Interpreter, Harness) and is powered end-to-end by **Amazon CloudWatch**, using **OpenTelemetry (OTEL)** as its underlying instrumentation standard via the AWS Distro for OpenTelemetry (ADOT). Because the instrumentation layer is OTEL rather than a proprietary schema, telemetry is natively exportable to any OTEL-compatible backend — AWS explicitly lists Arize Phoenix, Datadog, Dynatrace, Braintrust, Langfuse, and LangSmith as supported integration targets alongside CloudWatch itself.

What CloudWatch GenAI Observability surfaces natively: traces (full span hierarchy across model calls, tool invocations, memory reads/writes, gateway/policy decisions), session counts, latency and duration percentiles, token usage, error rates, and — since December 2025 — direct integration with **AgentCore Evaluations**, correlating the 13 built-in quality evaluators’ scores (helpfulness, tool-selection accuracy, correctness, safety) with the underlying prompts, tool calls, and logs in the same dashboard, plus Application Signals, Alarms, Sensitive Data Protection, and Logs Insights integration. Custom business metadata can be attached to traces, making observability legible to non-engineering stakeholders reviewing agent decisions.

What CloudWatch does *not* natively provide, per independent assessment: a dedicated LLM-evaluation *platform* comparable to Phoenix or Braintrust’s evaluation-first tooling — AWS’s own Evaluations service covers this to a meaningful degree as of 2026, but independent observability-landscape analysis (exploreagentics.ai, April 2026) still characterizes AWS’s strength as the *infrastructure-native, cloud-integrated* pick rather than the evaluation-experimentation-first pick.

13.2 24. Phoenix (Arize)

Phoenix is Arize’s open-source (Elastic License 2.0) AI observability and evaluation platform, vendor- and language-agnostic, with out-of-the-box auto-instrumentation (via the OpenInference project) for a wide range of frameworks and providers including AWS Bedrock directly. It can run entirely locally (zero cloud account, localhost:6006), in a notebook, containerized, or via Arize’s own hosted cloud instance (app.phoenix.arize.com). Because AgentCore is OTEL-compliant by construction, the integration pattern documented by both AWS and Arize is: build the agent (any framework, commonly Strands), package as a container, deploy to AgentCore Runtime, and point Runtime’s OTEL exporter at Phoenix — no code changes to the agent logic itself are required to add this second observability lane.

Phoenix’s differentiated strengths, per Arize’s own documentation and independent comparative reviews: **prompt/response lineage tracing** with full reasoning-chain visibility, **built-in evaluation** (LLM-as-judge scoring on dimensions like tool-calling correctness, with per-trace score attribution visible inline), drift detection, and cost analysis — capabilities that sit adjacent to, rather than duplicate, CloudWatch’s infrastructure-metrics strength.

13.3 25. Comparative Recommendation: CloudWatch-Only vs. Phoenix-Only vs. Hybrid

Dimension	CloudWatch Only	Phoenix Only	Hybrid (Recommended for most enterprise deployments)
Infra health (latency, errors, token cost)	Strong — native, zero extra setup	Weak — not its focus	CloudWatch
Reasoning-chain / trace-level debugging	Adequate	Strong — purpose-built UI for this	Phoenix
Quality evaluation (LLM-as-judge, drift)	Adequate (13 built-in evaluators via AgentCore Evaluations)	Strong — evaluation-first design	Phoenix for exploratory eval, CloudWatch for continuous production scoring
Compliance / audit trail, IAM-integrated access	Strong — native CloudTrail, Sensitive Data Protection	Weak unless self-hosted with matching controls	CloudWatch
Vendor neutrality / multi-cloud portability	Weak — CloudWatch is AWS-only	Strong — runs anywhere, OTEL-native	Phoenix as the portable layer if a multi-cloud future is plausible
Setup cost	Zero (default-on with AgentCore)	Low if self-hosted; adds an operational component	Marginal — one exporter config, same OTEL stream

Recommended enterprise reference architecture: treat CloudWatch as the **system-of-record for infrastructure health, cost, and compliance audit** (it is already the default, zero-setup path and integrates with existing AWS-native alerting/IAM), and layer Phoenix (or another OTEL-compatible platform such as Braintrust if evaluation-driven deployment gating is the priority) as the **engineering-facing trace-debugging and evaluation workbench**. Because both consume the same OTEL stream Strands/AgentCore already emits, this is additive rather than a fork in instrumentation strategy — a pattern independently validated by multiple June 2026 practitioner write-ups describing exactly this “CloudWatch for infra, Phoenix for traces, LLM-as-judge for quality” three-layer model.

13.4 26. Distributed Tracing, Span Hierarchy, and Root-Cause Analysis

A single agent turn typically produces a span tree: session span → model-invocation span(s) → tool-call span(s) (each nested under the model turn that requested it) → Gateway request/response-interceptor spans → Policy evaluation span → downstream target span (Lambda/API/MCP). Correlation IDs propagate through the entire chain, including across a multi-agent hand-off (Graph/Swarm), which is what makes it possible to root-cause a failure that originated in a specialist agent three hops downstream from the entry point. AWS’s own **Agent Inspector** and **Optimization/failure-insights** capabilities (GA by mid-2026) are built directly on top of this span data: failure insights specifically targets **silent behavioral failures that produce no error signal** — the hardest class of production agent bug — by mining recurring patterns across hundreds of sessions and ranking them by prevalence, with continuous daily/weekly reports or targeted post-deployment investigations completing in minutes rather than requiring manual trace archaeology.

14 Part XIII — Security: Complete Threat Model

14.1 27. Threat Model Overview

Agentic systems introduce a genuinely new class of attack surface distinct from traditional application security: the “exploit” can be a sentence, delivered through any content the agent reads, not just through a direct network request. This section maps each major threat class to the specific AgentCore capability designed to mitigate it, and — critically — notes where independent research shows the mitigation is partial or was found deficient.

14.1.1 27.1 Prompt Injection

Threat. An attacker embeds instructions in content the agent will read as part of its task context — a retrieved document, a tool result, a PR title, a web page — rather than in the user’s direct input. If the agent treats retrieved content with the same trust as its system instructions, it may follow the injected instruction. OWASP ranks prompt injection the #1 vulnerability in its LLM Top 10. A documented April 2026 academic/industry demonstration (Guan et al., Johns Hopkins) hijacked Claude Code, Gemini CLI, and GitHub Copilot via instructions embedded in GitHub PR titles, exfiltrating CI secrets through PR comments — illustrating the pattern is not theoretical.

AgentCore mitigation. Bedrock Guardrails, integrated directly into Policy/Gateway as of the April-June 2026 wave, evaluates *inputs to gateway targets* for injection attempts at the Gateway layer — structurally outside the agent’s own context, so the agent cannot “reason around” a check it never sees. This is a materially stronger position than prompt-level defenses (system-prompt instructions telling the model to ignore injected commands), which remain bypassable by construction. However, Guardrails-at-Gateway defends the *tool-call boundary*; it does not prevent an already-injected instruction from influencing the agent’s *reasoning or final text response* if the injected content never triggers a Gateway-mediated tool call. Defense-in-depth guidance (hidekazu-konishi.com, April 2026) recommends layering an **input guardrail with denied-topic policies** seeded with concrete jailbreak patterns (role-reassignment phrases, system-prompt-extraction phrases, encoding-based bypasses, and known indirect-injection markers) in addition to, not instead of, Gateway-layer Guardrails.

14.1.2 27.2 Tool (Injection) Poisoning / MCP Poisoning

Threat. A malicious or compromised MCP server, or a tampered tool *description* (metadata the agent reads to decide when/how to use a tool, often invisible to the end user), redirects the agent toward unintended actions without any change to the agent’s own code. Documented real-world instances in 2026 include a malicious MCP package (postmark-mcp) that shipped fifteen clean releases to build trust before quietly adding an exfiltration line, and large-scale scans finding hundreds of internet-exposed MCP servers with zero authentication.

AgentCore mitigation. Gateway’s aggregation model means every MCP target an agent reaches through AgentCore should itself be a vetted, Registry-catalogued resource (once Registry’s approval workflow is used as intended) rather than an ad hoc, unvetted server URL. Cedar policy evaluation happens per **tool call**, not merely per tool *description* — so even if a description is misleading, the actual invoked action, its arguments, and the calling principal are still checked against policy at the moment of execution. Registry’s Approver persona and auto-approval-off-by-default posture is specifically the governance control against onboarding an unvetted or poisoned tool into the organization’s shared catalog in the first place.

14.1.3 27.3 Data Exfiltration

Threat. An agent with legitimate access to sensitive data is manipulated (via injection or a poisoned tool) into sending that data somewhere it shouldn't go — an external API, a DNS query, an unauthorized recipient.

AgentCore mitigation. Cedar policy can restrict which tools/destinations a given principal may reach and under what data conditions (the geography/data-residency pattern documented in Part IV.6 is a direct instance of this). RESPONSE interceptors can redact sensitive fields from tool results before they re-enter the agent's context. Guardrails specifically screens for sensitive-data exposure at the Gateway boundary. **However**, the independently-documented Code Interpreter Sandbox-mode DNS gap (Part IX.11.2) is a concrete, real-world instance where a documented isolation boundary against exfiltration was weaker than advertised — the corrective control (VPC-only network mode plus least-privilege IAM roles specifically for the interpreter) should be treated as a required hardening step, not an optional one, for any workload where exfiltration risk matters.

14.1.4 27.4 Cross-Agent Attacks (Multi-Agent Trust Boundaries)

Threat. In a Graph/Swarm/supervisor-worker topology, a compromised or manipulated specialist agent could attempt to escalate privilege, access another user's data, or feed poisoned output back up the chain to the supervisor, which may trust it uncritically.

AgentCore mitigation. Identity propagation (Part VII.8.6) means a specialist agent's downstream tool calls are still evaluated against the *original user's* identity claims, not an elevated system identity — a compromised specialist cannot gain access the original user didn't have, regardless of how many hops deep it sits. Each agent in the chain, if deployed as a separate Runtime, gets its own microVM-isolated execution boundary and its own workload identity, so a compromise of one specialist agent's process does not directly grant code-level access to a sibling agent's memory or filesystem.

14.1.5 27.5 Privilege Escalation / Confused Deputy

Threat. A shared, over-broadly-trusted execution role lets an unrelated AgentCore resource (in the same or a different account) assume permissions intended for a specific Gateway or Runtime.

AgentCore mitigation. As detailed in Part VII.8.4, this is directly guarded against via `aws:SourceArn/aws:SourceAccount` conditions scoped to the specific resource ARN in every execution-role trust policy AWS's own samples ship. This is a **configuration responsibility**, not an automatic platform guarantee — omitting the condition (a documented, easy-to-miss mistake) reopens the confused-deputy path via the shared `bedrock-agentcore.amazonaws.com` service principal.

14.1.6 27.6 Replay Attacks and Token Theft

Threat. A captured OAuth authorization code or bearer token is replayed by an attacker to impersonate a legitimate agent or user session.

AgentCore mitigation. Session-binding mechanisms in the 3LO callback flow (Part VII, DeepWiki-documented) tie the authorization code's completion to a whitelisted, pre-registered callback URL per workload identity, structurally preventing redirect hijacking. Token Vault-issued credentials are short-lived and scoped; the Workload Access Token pattern means raw user tokens are never forwarded downstream to third-party resource servers, limiting the blast radius of any single token's theft.

14.1.7 27.7 Secret Management

Threat. Long-lived credentials embedded in agent code, container images, or environment variables.

AgentCore mitigation. The Token Vault is the sanctioned pattern precisely to eliminate this class of risk — credentials for downstream resources are fetched at invocation time via `@requires_access_token/@requires_api_key` decorators (in Strands-on-AgentCore code) rather than hard-coded, and are encrypted at rest and in transit.

14.1.8 27.8 Runtime, Browser, and Code Interpreter Isolation — Summary Assessment

All three compute-hosting primitives (Runtime, Browser, Code Interpreter) share the Firecracker-microVM-per-session isolation model, which independent security researchers (BeyondTrust: *“we applaud AWS for providing strong isolation properties of a full KVM-based VM”*) rate favorably relative to “agent as a service” competitors offering weaker container-only isolation. The documented weak point is specifically **network policy within an otherwise well-isolated VM boundary** (the Sandbox-mode DNS gap) and **IAM-permission scoping independent of network isolation** (the Sonrai S3-reachability finding) — not the VM isolation boundary itself, which held up under scrutiny. The practical takeaway for a threat model: trust the hardware-level session isolation; **do not** trust “Sandbox” network-mode naming alone to mean zero egress, and always pair network-mode hardening with least-privilege IAM scoping on the specific execution role.

15 Part XIV — Production Architecture

15.1 28. Network Connectivity Patterns

AWS’s own networking blog (May 2026) documents four progressively hardened patterns for AgentCore Runtime; the same patterns generalize to Gateway and Browser/Code Interpreter:

1. **Public endpoint, public egress (default).** Fastest to stand up; both inbound and outbound traffic traverse the internet. Appropriate only for prototyping or genuinely public-facing agents with no sensitive backend access.
2. **VPC egress via ENI, public ingress.** AgentCore provisions ENIs in customer-specified subnets so the agent can reach private resources (RDS, internal APIs, on-prem via Direct Connect/VPN) — but the agent’s own inbound endpoint remains internet-reachable.
3. **PrivateLink ingress + VPC egress, public ingress blocked.** A resource-based policy condition (`aws:SourceVpce/aws:SourceVpc`) rejects any request not arriving via the customer’s own VPC endpoint; combined with pattern 2’s egress, this fully removes the public internet from both directions of agent traffic — except for OAuth IdP round-trips and Gateway-to-external-MCP-server calls, which still require internet egress unless those specific targets are themselves reachable via PrivateLink or a private connectivity path.
4. **Full isolation — no IGW/NAT.** Every AWS service call (ECR image pulls, CloudWatch Logs, X-Ray, Bedrock) routes through its own VPC endpoint; no traffic enters or leaves the VPC through the public internet under any circumstance. This is the correct target state for the highest-sensitivity workloads (regulated PII/PHI/financial data) and is explicitly called out by AWS as providing the highest level of network isolation available on the platform.

Three distinct PrivateLink endpoint types exist and must be provisioned separately: the **data plane** endpoint (`bedrock-agentcore`) for Runtime/Memory/Identity/built-in-tools invocation, the **control plane** endpoint (`bedrock-agentcore-control`) for Runtime/Memory management operations, and the dedicated **Gateway** endpoint (`bedrock-agentcore.gateway`).

15.2 29. Multi-Account and Multi-Region Patterns

- **Single account.** Simplest; appropriate for a single team or early-stage adoption. All AgentCore resources (Runtime, Gateway, Registry, Policy engines) live in one account, one region.
- **Multi-account (recommended enterprise pattern).** A platform/security account owns shared Gateway targets, Registry catalogs, and Policy engine templates; workload accounts own their own Runtime/Harness deployments and consume the platform account’s governed catalog via cross-account IAM roles. As documented in Part V, **Registry currently has no native cross-account federation** (April 2026 preview state) — this pattern today requires custom cross-account IAM role assumption and manual catalog synchronization rather than a native federation feature, a real operational gap worth planning around explicitly rather than assuming away.
- **Multi-region.** AgentCore Runtime, Gateway, Policy, and Identity are regional services; a genuinely multi-region deployment requires independently provisioned resources per region, with cross-region replication of Memory data and Registry catalogs handled at the application/IaC layer (there is no documented native cross-region Memory or Registry replication as of June 2026).

15.3 30. Disaster Recovery: Active-Active vs. Active-Passive

- **Active-passive.** A standby region holds provisioned-but-idle Runtime/Gateway/Policy resources, with Route 53 or Global Accelerator failover redirecting traffic on primary-region failure. Memory and Registry state require an explicit replication mechanism

(e.g., DynamoDB Global Tables if state has been externalized there, or scheduled Registry-record export/import) since neither service natively replicates cross-region.

- **Active-active.** Both regions serve live traffic; session affinity becomes region-scoped (a session started in one region should not expect its microVM state to be reachable from the other), which pushes any genuinely cross-region session continuity requirement onto an externalized state store with its own cross-region replication (DynamoDB Global Tables, Aurora Global Database, or a cross-region cache).
- The consumption-based, serverless nature of Runtime/Gateway/Harness means a standby region's *idle* cost is materially lower than a comparable EC2/EKS active-passive DR posture — no idle compute capacity needs to be paid for beyond Memory storage and any pre-warmed session pool a team chooses to maintain (Part III.3.3).

15.4 31. Hybrid Cloud

VPC connectivity plus Direct Connect/VPN lets AgentCore Runtime, Browser, and Code Interpreter reach on-premises systems as if they were private VPC resources, without exposing those systems to the public internet. The identity-federation limitation noted in Part VII.8.5 (no native cross-cloud agent-identity unification with Azure/GCP) is the primary architectural gap for genuinely hybrid or multi-cloud agent estates; a third-party identity-federation layer, or a deliberate choice to run cloud-specific agent fleets per provider with governed hand-offs at defined boundaries, are the two practical mitigations as of mid-2026.

16 Part XV — Release Analysis: April-June 2026

16.1 32. Chronological Timeline

Date	Release	Significance
March 3, 2026	Policy reaches GA	Deterministic, Cedar-based, Gateway-enforced authorization becomes a production-ready control, not a preview feature
March 31, 2026	Evaluations reaches GA	13 built-in evaluators; continuous quality monitoring becomes standard, integrated with CloudWatch
~April 2026	Registry enters public preview	Governance catalog for agents/tools/skills; four-persona IAM model; five regions; free during preview
April 22, 2026	Harness enters public preview	Two-API-call agent definition; Strands-powered; no orchestration code required
April 28, 2026	AWS “What’s Next” event	Managed harness (preview), AgentCore CLI (IaC-governed deployment, CDK at launch, Terraform later in 2026), AgentCore skills for coding assistants
~May 2026	Optimization (recommendations + failure insights) reaches GA	Production-trace-driven prompt/tool-description recommendations; silent-failure pattern mining across sessions
May 7, 2026	Payments enters preview	Agent-autonomous transacting via x402 protocol; Coinbase CDP and Stripe Privy wallet connections; four regions
~May-June 2026	Gateway: Runtime-as-target GA	Gateway can front a Runtime agent directly as an HTTP target with API-schema-driven Guardrails application
June 1, 2026	AWS blog: Policy + Lambda interceptor composability patterns	Formalizes the three-pattern design guidance (policy-only, interceptor-only, combined) for production security architecture
June 17, 2026	AWS Summit New York — AWS Continuum and AWS Context previewed	Adjacent (non-AgentCore) platform services signaling AWS’s “trust ladder” investment direction (see Part XVI)

Date	Release	Significance
June 18, 2026	Harness reaches GA	Multi-model mid-session switching; auto-provisioned managed memory; A/B testing; Step Functions integration; export to Strands code
Ongoing (June 2026)	Runtime quota increases	Active sessions per account raised to 5,000 (us-east-1/us-west-2) / 2,500 (other regions); InvokeAgentRuntime rate raised from 25 to 200 TPS per agent per account

16.2 33. Feature-Level Deep Dives

16.2.1 33.1 Policy GA — Migration Guidance

Teams that built custom in-agent-code authorization logic before Policy’s GA should migrate incrementally: stand up a Policy engine in **LOG_ONLY** mode alongside existing controls, validate that Cedar decisions match the legacy logic’s expected outcomes against real traffic, then cut over to **ENFORCE** and only *then* remove the legacy in-code checks. Removing in-code checks before ENFORCE validation is complete leaves a window with no authorization at all.

16.2.2 33.2 Harness GA — Migration Guidance

Teams that built early prototypes directly on Runtime with hand-written Strands orchestration should evaluate whether Harness’s configuration-only model now covers their use case — the GA “export to Strands code” feature is explicitly designed to make this a **two-way**, not one-way, decision: start on Harness for speed, export to full Strands/Runtime control if and when the configuration surface becomes limiting, rather than needing to choose irreversibly at project start.

16.2.3 33.3 Registry Preview — Adoption Guidance

Given the documented preview-stage limitations (Part V.6.3), the responsible adoption pattern in June 2026 is: use Registry now for the low-risk, high-value use case it already supports well — a build-time, human-governed catalog of *internal* skills and vetted MCP tools within a single account — while explicitly not yet depending on it for cross-account federation or IaC-managed provisioning, both of which are roadmap items rather than current capabilities.

16.2.4 33.4 Payments Preview — Risk Framing

Payments is explicitly preview-only and should not be treated as a production payment rail. AWS’s own design channels risk through **deterministic, infrastructure-layer spend limits** (set at the session level) rather than trusting the agent’s own reasoning to self-limit spend — consistent with the platform’s overall philosophy (Part V of the Threat Model: enforcement outside the agent’s reasoning, not inside it). The Coinbase x402 Bazaar MCP server integration (10,000+ payable endpoints) previews a genuinely new category of agent-to-service commerce, but four-region availability and preview status mean production dependence should

wait for GA and a second look at independent security research once broader adoption produces a research target.

17 Part XVI — Roadmap Prediction

17.1 34. Method

Predictions below are grounded exclusively in public evidence gathered for this report: AWS release notes and blogs, GA/preview timing patterns observed April-June 2026, AWS Summit/re:Invent announcements, GitHub activity in the strands-agents organization, and adjacent AWS platform announcements (Continuum, Context) that signal strategic direction. Each prediction carries a confidence level (**High** / **Medium** / **Low**) and its evidentiary basis. Predictions are explicitly forward-looking and unverifiable at time of writing — they are structured inference, not confirmed roadmap.

17.2 35. Next 6 Months (through ~December 2026)

Prediction	Confidence	Evidence
Registry reaches GA , adding Terraform/CloudFormation support and at least basic cross-account sharing	Medium	Registry moved preview→visible-roadmap-item within ~2 months of launch (April→June mentions of “additional connectors” at the June summit); AWS’s historical pattern (Policy: preview→GA in similar-order months; Evaluations: preview→GA in ~4 months) suggests a GA push within 6–9 months of an April 2026 preview start, but no explicit GA date has been stated
Payments reaches broader regional availability , still likely preview	Medium	Four regions at May 2026 launch is narrow even by AgentCore preview standards; the partnership structure (Coinbase, Stripe) and cited Bazaar MCP server scale (10,000+ endpoints) suggest AWS intends rapid expansion, but agent-commerce is a genuinely novel risk category AWS is likely to expand cautiously
Harness gains a native HTTP/MCP/A2A endpoint , closing the API-only gap noted at preview	Medium	The gap was explicitly noted by independent reviewers at preview (April 2026); GA already added Step Functions integration and code export, both signals of AWS actively closing integration gaps quickly post-preview

Prediction	Confidence	Evidence
Policy gains additional third-party detection-signal integrations (Check Point, Zscaler, Rubrik, Netskope, SentinelOne — all explicitly named “coming soon” by AWS)	High	Directly and explicitly stated by AWS in an official blog as a near-term commitment, not inferred
Code Interpreter Sandbox-mode network isolation is hardened further , following the DNS-exfiltration disclosure and partial fix	High	AWS already shipped one remediation (DNS exfiltration blocked) within roughly five weeks of BeyondTrust’s disclosure; the Sonrai IAM-reachability finding (S3 access from Sandbox mode) remains open at time of writing and is a logical next target given the pattern of rapid response to credible external security research

17.3 36. Next 12 Months (through ~mid-2027)

Prediction	Confidence	Evidence
A Policy Studio / visual policy-authoring console emerges, building on the existing NL2Cedar neuro-symbolic pipeline	Medium	The natural-language-to-Cedar pipeline and Cedar Analysis are already GA-quality building blocks; AWS has a strong historical pattern (IAM Access Analyzer, Verified Permissions console) of eventually wrapping policy-as-code capabilities in a visual authoring layer once the underlying engine matures

Prediction	Confidence	Evidence
Native multi-agent orchestration at the AgentCore platform level (beyond Strands' SDK-level Graph/Swarm and Gateway's A2A passthrough) — e.g., a managed “Orchestrator” service	Medium	S&P Global Market Intelligence’s public case study explicitly cited struggling to orchestrate complex multi-agent workflows and needing a unified memory layer, which AgentCore Memory partially addressed; the <i>orchestration</i> half of that gap (not just shared memory) remains a plausible next platform-level service, especially given AWS Context’s knowledge-graph direction as adjacent infrastructure Explicitly named as a preview-stage gap by AWS’s own early adopters; federation is architecturally harder than the GA items above and typically lags initial GA by 2+ release cycles in comparable AWS catalog services (e.g., cross-account Resource Access Manager patterns took multiple years to mature for other AWS catalogs) Consistent with AWS’s stated positioning (“stay aligned with numerous AWS compliance programs”) and the general AWS pattern of shipping compliance-program-mapped configuration bundles (e.g., Config Conformance Packs) once a policy engine matures; no explicit AgentCore-specific announcement found
Registry federation (cross-account, possibly cross-region) ships	Low-Medium	
Compliance Packs (pre-built Cedar policy + Guardrails bundles mapped to specific regulatory frameworks — HIPAA, PCI-DSS, FedRAMP)	Medium	

Prediction	Confidence	Evidence
Evaluation Studio (a dedicated, richer authoring/comparison UI for custom evaluators, closing the gap with Phoenix/Braintrust's evaluation-first UX)	Medium	Evaluations and Optimization both reached GA within the report window and are clearly an active investment area; independent observability-landscape analysis explicitly frames evaluation-platform depth as AWS's current relative weak point versus Phoenix/Braintrust, which is the kind of competitive gap AWS has historically closed within 12-18 months once identified
Cost Optimizer (automated right-sizing recommendations for warm pools, memory strategy selection, model routing)	Medium	The Optimization service's existing "recommendations" capability already analyzes production traces to suggest prompt/tool-description fixes; extending the same trace-mining approach to cost (not just quality) is a natural, low-lift extension of infrastructure AWS has already built

17.4 37. Next 24 Months (through ~mid-2028)

Prediction	Confidence	Evidence
Enterprise Agent Marketplace (cross-organization, not just cross-account within one enterprise) reaches meaningful adoption	Medium	Explicitly named as a roadmap item at re:Invent 2025 ("a marketplace where teams can publish or subscribe to reusable agent capabilities"); the AWS Marketplace "AI Agents & Tools" solution page already exists as of early 2026 for partner-published offerings, but a genuine AgentCore-native, Registry-integrated marketplace (vs. today's general AWS Marketplace listing mechanism) is a larger integration lift likely to land later in this window

Prediction	Confidence	Evidence
Agent Lifecycle Manager (a unified deprecation/promotion/rollback console spanning Registry + Harness/Runtime versioning)	Medium-High	The individual building blocks (Registry lifecycle states, Runtime/Harness immutable versioning with named-endpoint rollback) already exist independently; unifying them into one lifecycle-management surface is a coherent, low-novelty integration of existing primitives rather than new invention
Visual Agent Builder (no-code/low-code agent construction UI, likely harness-backed)	Low-Medium	AWS’s overall product philosophy across this report — CLI-first, code-exportable, “export to Strands code” as an explicit design principle at Harness GA — suggests AWS is more likely to keep deepening the code-first/config-first Harness path than to invest heavily in a separate no-code builder; Bedrock’s existing (separate, older) visual Agent Builder for classic Bedrock Agents suggests appetite exists, but AgentCore’s stated positioning has been consistently code/config-first to date
Governance dashboards unifying Policy decisions, Registry approval workflows, and Payments spend across an entire organization into one executive-facing view	Medium	A direct, logical extension of AWS Continuum’s demonstrated “learn mode → enforce mode” staged-trust UX pattern (already shipping for code-vulnerability remediation as of June 2026) applied to the agent-governance domain; AWS has explicitly signaled cross-product coherence around “trust ladders” as a company-wide theme at the June 2026 summit

Prediction	Confidence	Evidence
AgentCore Payments reaches GA with expanded regional coverage and additional wallet/rail partners beyond Coinbase/Stripe	Medium	Novel risk category (Part XV.33.4) makes a cautious, multi-quarter preview-to-GA timeline likely rather than a fast-track; 24 months is a conservative but reasonably confident window given the partnership investment already made
Deeper convergence with AWS Context (the identity-aware enterprise knowledge graph previewed June 2026) as a native AgentCore Memory/Gateway data source	Medium	Context and AgentCore Memory address adjacent but distinct problems (organization-wide business knowledge vs. per-agent conversational memory); AWS's own framing at the June summit positioned Context as available "to every agent across an organization," which logically implies an AgentCore integration path even though none was explicitly announced within the report window

18 Part XVII — Adjacent Roadmap Signal: AWS Continuum and AWS Context

Though not AgentCore services themselves, both were previewed at the same June 17, 2026 AWS Summit New York event alongside AgentCore enhancements, by the same leadership (VP of Agentic AI Swami Sivasubramanian, Chief AI and Technology Officer Matt Wood), and are directly relevant to any AgentCore roadmap forecast because they reveal AWS’s company-wide thesis for the next investment cycle: **trust, not raw capability, is the bottleneck to enterprise agent adoption.**

AWS Continuum is an AI-native security service for autonomous code-vulnerability discovery, prioritization, validation, and remediation. Architecturally, it demonstrates the **staged-trust pattern** independent analysis expects AWS to generalize across its agent portfolio: it launches in “learn mode” (every recommendation surfaces with full reasoning and an audit trail; no autonomous action), and organizations promote specific finding *categories* to “enforce mode” individually as confidence is earned — never an all-or-nothing autonomy switch. Exploit validation happens inside isolated sandboxes before any fix is proposed, filtering false positives that would otherwise generate noisy remediation PRs. This launched partly in direct response to internally-reported incidents where AI-generated code changes caused AWS’s own production outages in February 2026, which is itself a useful, independently-reported data point on the real-world failure modes agentic systems produce at scale.

AWS Context auto-builds an identity-aware knowledge graph from an organization’s databases, documents, and chat history, making it queryable and discoverable by every agent across the organization, with access itself scoped by the querying identity’s existing permissions (so an agent cannot surface information via Context that the underlying identity wouldn’t otherwise be authorized to see). AWS’s own framing positions Context as solving the fourth-generation problem in the agent-infrastructure stack — after compute (Runtime), tools (Gateway), and governance (Policy/Registry) comes *business knowledge* — and independent analysis (TechTarget, June 2026) notes AWS is arriving at this “context layer” concept after several other vendors had already begun shipping similar knowledge-graph offerings, making AWS a fast-follower on this specific capability rather than a first-mover.

Why this matters for the AgentCore roadmap specifically: both services validate the predictions in Part XVI that emphasize *staged, auditable autonomy* (Governance dashboards, Compliance Packs) and *deeper business-context integration* (the Context-to-Memory convergence prediction) as the most evidence-backed near-term investment areas, rather than raw capability expansion.

19 Part XVIII — Best Practices

19.1 38. Runtime and Harness

- Externalize any state an MCP server or long-running agent needs to survive a microVM recycle — never rely on in-process RAM as a correctness mechanism, only as a latency optimization.
- Use named endpoints (dev/staging/prod) rather than always deploying to DEFAULT, to get zero-downtime rollback via endpoint repointing rather than redeployment.
- Right-size warm pools against measured traffic patterns rather than either accepting cold-start latency or over-provisioning a static pool.
- Start new projects on Harness for speed; explicitly plan the “export to Strands code” exit ramp before configuration complexity outgrows what Harness exposes, rather than discovering the ceiling mid-project.
- Disable Strands hot-reload in any container destined for Runtime or Harness — treat it as strictly local/dev tooling.

19.2 39. Gateway and Policy

- Always start a new Policy engine in **LOG_ONLY** and validate against real production traffic before flipping to **ENFORCE**.
- Treat natural-language-generated Cedar as a draft requiring human review against the actual Gateway schema, never as production-ready output on first generation.
- Use REQUEST interceptors for anything dynamic (token exchange, external context lookups); use Cedar for anything expressible as a static rule over the resulting context — do not conflate the two responsibilities.
- Scope every execution-role trust policy with `aws:SourceArn/aws:SourceAccount` conditions bound to the specific Gateway/Runtime ARN, without exception.
- Do not remove IAM-layer scoping under the assumption that Policy supersedes it — the two layers answer different questions and both must be independently correct.

19.3 40. Identity

- Prefer OBO token exchange over raw token passthrough wherever the downstream service supports it — it preserves both user and agent identity for fine-grained downstream authorization without a re-consent prompt.
- Register callback URLs per workload identity explicitly; never widen the whitelist beyond what a specific agent’s OAuth flows actually require.
- For genuinely multi-cloud agent estates, evaluate a dedicated cross-cloud identity-federation layer early — do not assume AgentCore Identity alone will unify identity across AWS, Azure AI Foundry, and Vertex AI.

19.4 41. Memory

- Match memory strategy to retrieval contract, not to strategy name popularity — Semantic is not automatically the right default; a travel-preference use case wants User Preference, a workflow-audit use case wants Episodic.
- Set short-term event-expiry deliberately per use case rather than accepting a default; regulated data may need materially shorter retention than the 30-day Harness-GA default.
- Rely on namespace-based (actorId-keyed) isolation as the multi-tenant boundary, not application-layer filtering after retrieval.

19.5 42. Code Interpreter and Browser

- Treat “Sandbox” network mode as *reduced*, not *zero*, network exposure — DNS-tunneling-class exfiltration was independently demonstrated and only partially remediated by AWS at time of writing (the underlying IAM-role-reachability finding from Sonrai remains open).
- Use VPC-only network mode for any Code Interpreter session touching sensitive data, and pair it with a tightly-scoped, dedicated execution role — never a role shared with other AgentCore primitives.
- Enable full session replay and CloudTrail logging on Browser sessions handling any authenticated or sensitive web interaction, treating it as an auditable action a human may need to review after the fact.

19.6 43. Observability

- Default to CloudWatch for infrastructure health, cost, and compliance audit; layer Phoenix (or an equivalent OTEL-compatible platform) for trace-level debugging and evaluation-driven development — treat this as additive instrumentation, not a fork.
- Use Optimization/failure-insights’ continuous monitoring specifically to catch **silent** behavioral failures (no error signal, wrong or subtly-degraded output) — the class of bug that infrastructure metrics alone will never surface.

20 Part XIX — Anti-Patterns

- **Treating Policy as a replacement for IAM**, rather than a complementary layer — removes an independent line of defense.
- **Relying on session-ID stickiness for MCP protocol-level correctness** — it is a routing optimization, not a durability guarantee, and will break under real scaling events.
- **Assuming Registry preview auto-catalogs deployed Runtime agents** — it does not; registration is a manual, explicit step teams must build into their deployment pipeline themselves.
- **Assuming “Sandbox” network mode in Code Interpreter means zero egress** — independently disproven; DNS-based exfiltration was demonstrated, and IAM-role reachability from within Sandbox mode is a separate, still-open finding.
- **Leaving Strands hot-reload enabled in a production container** — turns a convenience feature into an unsandboxed arbitrary-code-execution surface.
- **Widening OAuth callback-URL whitelists broadly “to be safe”** — inverts the actual security property the whitelist exists to provide.
- **Deploying multi-region without an explicit state-replication plan for Memory and Registry** — neither replicates natively cross-region; assuming otherwise produces a DR plan that fails exactly when it’s needed.
- **Skipping the LOG_ONLY validation phase before ENFORCE** — the single most commonly cited practitioner mistake across the Policy adoption write-ups reviewed for this report; it converts a legitimate business call into an outage.
- **Treating a single quote/estimate from AWS marketing (spend caps, “15× performance,” auto-learning claims) as independently verified fact** — several such figures at the June 2026 summit were explicitly flagged by independent analysts as vendor-stated and awaiting third-party verification; architects should distinguish AWS-documented behavior from AWS-marketed claims when making dependency decisions.

21 Part XX — Production Readiness Checklist

- ☐ Every Gateway has an explicit inbound authorization mode configured (never “no authorization” outside dev/test)
- ☐ Every outbound target uses a scoped credential pattern (2LO/3LO/OBO/IAM), never “no authorization”
- ☐ Every execution-role trust policy includes `aws:SourceArn/aws:SourceAccount` conditions
- ☐ Policy engine has completed a LOG_ONLY validation window against real traffic before ENFORCE cutover
- ☐ Cedar policy set has passed Cedar Analysis with no unresolved conflicts/redundancies
- ☐ Bedrock Guardrails attached at the Gateway layer for prompt-injection and sensitive-data-exposure screening
- ☐ Code Interpreter sessions handling sensitive data run in VPC-only network mode with a dedicated, least-privilege execution role
- ☐ MCP servers hosted on Runtime externalize all protocol-level session state (no reliance on in-process RAM)
- ☐ Strands hot-reload is disabled/absent in the deployed container
- ☐ Memory short-term event expiry and long-term strategy selection are deliberate choices, documented per use case
- ☐ Multi-region deployments have an explicit, tested Memory and Registry state-replication or resynchronization plan
- ☐ OTEL export is configured to both CloudWatch and at least one secondary trace-debugging platform for engineering-facing visibility
- ☐ Failure-insights/continuous-monitoring is enabled to catch silent (no-error-signal) behavioral regressions
- ☐ A documented incident-response playbook exists specifically for prompt-injection and tool/MCP-poisoning scenarios (see Part XXII)
- ☐ Registry entries for all production agents/tools have gone through the Approver persona’s review, with auto-approval left off

22 Part XXI — Cost Optimization Guide

- **Warm pools over blanket pre-provisioning.** AgentCore’s memory-vs-compute pricing split makes maintaining a right-sized warm pool of idle (memory-billed-only) microVMs cheaper than either accepting cold-start latency or provisioning full active compute capacity ahead of demand — monitor actual traffic and adjust pool size by time-of-day.
- **Match memory strategy cost to actual retrieval need.** Self-managed memory strategies are billed differently (and generally at a premium) versus built-in strategies — reach for a built-in strategy first and only build custom extraction/consolidation logic when a built-in strategy’s output schema is genuinely insufficient.
- **Use Optimization’s recommendation engine before hand-tuning.** It mines production traces to suggest concrete, grounded prompt and tool-description fixes rather than requiring manual trial-and-error — a lower-cost path to quality improvement than iterative human-driven prompt engineering alone.
- **Right-size Code Interpreter session duration.** The default 15-minute execution window is appropriate for most tasks; only extend toward the 8-hour maximum for workloads that genuinely require it, since longer-running sandboxes hold billed resources for longer.
- **Treat model inference, not AgentCore infrastructure, as the primary cost lever.** Industry estimates place AgentCore’s own infrastructure cost at roughly 10–30% of total agent spend at scale — cost-optimization effort is generally better spent on model selection/routing (Harness’s per-invocation model override enables cheap-model-for-simple-steps, expensive-model-for-hard-steps routing within a single session) than on infrastruc-

ture tuning alone.

23 Part XXII — Security Hardening Guide (Summary)

1. Layer defenses in series: adaptive-retry-aware error handling → input Guardrails with seeded jailbreak/injection patterns → Cedar policy at the Gateway → least-privilege IAM on every execution role → VPC-only network mode for sensitive compute → full audit logging (CloudTrail, Observability traces) as the backstop that makes every other layer reviewable after the fact.
2. Apply a WAF at the actual internet-facing front door (CloudFront/API Gateway/ALB) — AgentCore Runtime itself does not host a WAF, so this layer must be added explicitly by the deploying team if public ingress is retained.
3. Never assume a single security control is sufficient; the recurring theme across every independent finding in this report (Code Interpreter Sandbox mode, confused-deputy trust-policy gaps, Registry auto-approval) is that AWS ships a *documented* safe default, but the *effective* security posture depends on the deploying team correctly configuring — and periodically re-validating — every layer, not just enabling the platform feature and assuming it is sufficient on its own.
4. Build an incident-response playbook specifically for agent-native failure modes: a prompt-injection incident should have a defined process for (a) identifying which Gateway-mediated tool calls occurred during the affected session via Observability traces, (b) determining whether Guardrails or Policy caught/blocked the injected action, (c) revoking any Token Vault credentials the affected session held, and (d) reviewing whether the injected content entered via a Registry-catalogued (vetted) or unvetted tool/MCP source, since that materially changes the remediation (patch a specific tool vs. re-review the entire unvetted-source policy).

24 Part XXIII — Keeping the Agent Live: Resilience, Roll-out, and Control Operability

This part addresses the operational control surface a platform team needs to run agents continuously in production: how to load-test them, how to stop them when they misbehave, how to roll out changes safely, how to pause for a human, how to resume after a long wait, and how to tell an auth failure apart from a business-rule denial. Every mechanism below is either an AWS-documented AgentCore capability or an independently-documented gap/workaround — both are marked explicitly, because the gaps matter as much as the capabilities for an architect making a build-vs-buy call on the control plane.

24.1 44. Stress Testing and Load Testing

Agent load testing is qualitatively different from conventional API load testing, and the difference matters for capacity planning: a $2\times$ increase in concurrent sessions does not produce a proportional latency increase, because token-generation cost, tool-call fan-out, and context-window growth all compound non-linearly. Independent load-testing analysis (LoadView, GetVocal) converges on measuring **latency elasticity** — how response time bends as concurrency rises — rather than flat requests-per-second, and on tracking **cognitive-load failure modes** distinct from infrastructure failure: hallucination rate, tool-call failure rate, and task-completion rate all typically degrade *before* raw latency or error-rate thresholds are breached, meaning an agent under load can look “green” on infrastructure dashboards while quietly answering worse.

AgentCore-specific load-testing surface: - **Quota-aware test design.** As of the June 2026 quota increases, InvokeAgentRuntime supports up to 200 TPS per agent per account (25 TPS previously), active sessions up to 5,000 per account in us-east-1/us-west-2 (2,500 elsewhere), and container-deployment session creation up to 400 TPM per endpoint. A load test plan should explicitly target these ceilings — both to validate real headroom and to confirm ServiceQuotaExceededException handling degrades gracefully rather than cascading. - **AgentCore Evaluations, batch mode,** is the AWS-native mechanism for regression-style load/quality testing: run the agent against a curated dataset in batch, score aggregate results against a baseline, and — per AWS’s own guidance — wire this into CI/CD so no configuration change reaches production without passing known-good cases first. This is complementary to, not a replacement for, raw throughput load testing. - **A documented idle-timeout footgun directly relevant to load-test design:** if a custom /ping handler sets `time_of_last_update` to the current time on every ping (rather than only when status genuinely changes), the platform’s idle-timeout calculation never fires, sessions accumulate past their intended idle window, and a sustained test run can silently exhaust the account’s session quota via `maxVms/ServiceQuotaExceededException` — a bug in test harness code, not the platform, but one specifically triggered by the ping-based health model AgentCore uses. - **Third-party tooling** (k6, Locust, Gatling, or agent-specific harnesses like Botium) remains necessary for raw concurrency/throughput testing; AgentCore does not ship a load-generation tool itself, only the evaluation/observability surface to interpret results against.

24.2 45. Kill Switch

What AgentCore provides natively: `StopRuntimeSession` — a documented API (boto3 `stop_runtime_session` or the equivalent HTTP endpoint) that immediately terminates a specific active session by `agentRuntimeArn + runtimeSessionId`, halting any in-flight streaming response and releasing the microVM. This is the sanctioned, single-session kill switch, and AWS’s own `re:Post` guidance recommends calling it explicitly as part of timeout/error-handling logic, not only as a manual emergency action.

A documented, material gap (independent finding, GitHub issue #498 against `aws/bedrock-agentcore-starter-toolkit`, April 2026): there is **no API to list or bulk-terminate all active sessions** — an operator facing a fleet-wide runaway cannot enumerate and kill every session in one call; `list-sessions` only lists *Memory* sessions, not *Runtime/microVM* sessions, which is a specific, easy-to-make confusion under incident pressure. The same report documents a worse edge case: a genuinely looping agent that continuously writes to its persistent workspace volume can prevent `StopRuntimeSession`'s own shutdown sequence from completing, because `AgentCore` attempts to back up the volume before killing the container, and a continuously-written volume never finishes backing up — the stop request effectively stalls. The only workaround the reporting team found effective was an **out-of-band IAM emergency-deny policy** attached directly to the runtime's execution role, denying `bedrock:InvokeModel / InvokeModelWithResponseStream / CallWithBearerToken`, which starves the agent's next model call with a 403 and breaks the loop from outside the agent entirely — consistent with this report's recurring theme that effective containment lives outside the agent process, never inside it.

Practical kill-switch architecture, synthesizing AWS's primitive and the independent gap-fill patterns:

1. **Per-session:** `StopRuntimeSession`, called proactively on timeout/ping-failure/error conditions in your own orchestration code — do not wait for a human to notice.
2. **Fleet-wide / emergency:** maintain your own session registry (log every `runtimeSessionId` you create, e.g., to `DynamoDB`, at invocation time) specifically *because* `AgentCore` does not expose a native list-and-kill-all API; treat this as a required piece of your own control plane, not an optional nicety.
3. **Last-resort, when a session is unresponsive to `StopRuntimeSession`:** a pre-staged, single-purpose IAM emergency-deny policy scoped to the specific runtime's execution role (per the confused-deputy guidance in Part VII, this role should already be scoped to exactly one runtime, which is also what makes an emergency-deny safe to apply without collateral impact on other workloads). Know the role name and have the deny-policy JSON ready *before* an incident, not during one.
4. **Governance-layer kill switch (pre-execution, not post-hoc):** a Cedar forbid policy attached in `ENFORCE` mode is itself a kill switch for a specific tool, principal, or action class — flipping a single policy from permit to forbid (or adding a global forbid) blocks the dangerous action at the Gateway before it ever reaches the tool, which is faster and safer than terminating a session already mid-execution.

24.3 46. Circuit Breakers

A circuit breaker differs from a kill switch in scope and trigger: a kill switch is a manual or session-scoped stop; a circuit breaker is an **automated, threshold-driven** trip — iteration count, dollar/token budget, consecutive failure count, or a semantic policy violation — that halts *before* a human notices, and (ideally) recovers automatically once the underlying condition clears. The independent tooling landscape here (`Waxell Runtime`, and several open-source Show-HN-era projects — `AgentFuse`, `AgentCircuit`, `FailWatch`, `Runtime Fence`) exists specifically because, as of mid-2026, **no cloud agent platform, AgentCore included, ships a first-class, configurable circuit-breaker primitive as a single toggle** — this is a genuine gap independent analysis converges on, not an `AgentCore`-specific shortcoming.

What `AgentCore` *does* give you to assemble an equivalent, distributed across the layers already documented in this report:

- **Cedar forbid rules with context conditions** (Part IV) can encode budget-style logic directly — e.g., deny a tool call when `context.input.amount` or a running total exceeds a threshold — evaluated deterministically at the Gateway, outside agent reasoning, which is exactly the “enforcement lives outside the agent” property a circuit breaker needs.
- **Lambda REQUEST interceptors** can maintain per-session or per-principal counters (iteration count, cumulative spend) in an external store (`Redis/DynamoDB`) and inject a context attribute Cedar then evaluates — this is the composable pattern from Part IV.6 applied specifically to circuit-breaking rather than access control.
- **`add_async_task/complete_async_task` plus the `/ping HealthyBusy` contract** give you a

natural hook point to enforce a hard wall-clock or iteration ceiling on long-running work: a custom ping handler can inspect elapsed time or task count and simply stop reporting HealthyBusy, which surfaces the stall through normal Runtime health-checking. - The explicit architectural conclusion independent analysis reaches, and this report concurs with: **circuit-breaking should be a first-class infrastructure primitive, not bespoke per-team code** — until AgentCore ships one natively, budget this as platform-team-owned shared infrastructure (a small Lambda interceptor + Cedar policy pair, reused across every Gateway) rather than letting each agent team reinvent it.

24.4 47. Canary Release and Progressive Rollout

AgentCore does **not** offer Lambda-style weighted-alias traffic splitting directly on Runtime endpoint versions (there is no routing-config equivalent for AgentCore Runtime endpoints the way there is for a Lambda alias). Progressive rollout on AgentCore is instead achieved through two AWS-documented, Gateway-mediated mechanisms:

1. Gateway HTTP passthrough targets with session-sticky weighted routing. Gateway now supports (per the release-notes wave covered in Part XV) HTTP passthrough targets that front any HTTP endpoint — including another AgentCore Runtime version, an external agent, or an A2A service — with configurable **session stickiness so weighted routing rules keep a given session on the same target** once assigned. This is the closest AgentCore analog to a canary weight, applied at the Gateway rather than the Runtime layer.

2. AgentCore Optimization’s A/B testing (GA-adjacent, part of the Optimization/agent-performance-loop capability). This is the fully-documented, purpose-built mechanism, and it is worth treating as *the* canary/progressive-rollout primitive for AgentCore rather than reaching for hand-rolled weighted routing: - An A/B test defines **control** and **treatment** variants two ways: **target-based routing** (different named Runtime endpoints, registered as separate Gateway targets — use this when the change involves code, a framework upgrade, or an entirely different implementation) or **configuration-bundle-based routing** (same Runtime, different immutable configuration-bundle version — use this when the change is purely a system prompt, model ID, or tool-description edit, requiring no redeployment at all). - Gateway assigns each **session** (not each request) to a variant based on the X-Amzn-Bedrock-AgentCore-Runtime-Session-Id header and the configured traffic weights; **assignment is sticky** — once a session lands on a variant, every subsequent request in that session stays on it, giving within-session consistency while still distributing new sessions per the configured split. - **Online evaluation scores every session automatically** against configured evaluators, and the results object — polled via GetABTest — reports, per evaluator, mean score, absolute and percent change, **p-value, confidence interval, and an isSignificant boolean** (AWS’s stated threshold: $p < 0.05$). Results can be polled at any time without affecting statistical validity. - **Promotion and rollback are explicit, first-class operations:** agentcore promote ab-test stops the test, repoints the control endpoint to the treatment version (or updates the winning gateway target), and removes the losing variant, after which agentcore deploy applies the change — a clean, auditable promotion step rather than a manual traffic-weight edit. Pausing an in-flight test simply reverts all traffic to the existing (control) configuration, which is itself a rollback mechanism with no additional steps. - Config-bundle-based A/B tests are notably cheap to run: a documented example ran a 50/50 split entirely through configuration versioning, “with no container rebuild,” which is a materially faster iteration loop than infrastructure-level canarying.

Practical guidance: default to configuration-bundle A/B testing for prompt/model/tool-description changes (fast, cheap, no redeploy), and reserve target-based (Gateway-routed, separate-Runtime) A/B testing for genuine code or framework changes. In both cases, do not promote until the target evaluator reports `isSignificant: true` with a positive `percentChange` — the practitioner write-ups reviewed for this report show real cases (a +18%

directional lift at $p=0.059$) that *look* like wins but fail the significance bar at low sample sizes, and promoting on a directional-but-not-significant result is a documented practitioner mistake worth guarding against explicitly.

24.5 47a. Feature Gates

Two distinct feature-gating mechanisms apply to AgentCore agents, at two different layers:

AWS-native: Configuration Bundles as a gating primitive. Beyond their role in A/B testing (Part 47), configuration bundles are a general-purpose feature-gate mechanism in their own right: a git-like versioned, branch-organized, immutable snapshot of any key-value configuration (system prompt, model ID, tool descriptions, or arbitrary custom keys an agent chooses to read) that the agent’s own code reads dynamically at invocation time via `Bedrock-AgentCoreContext.get_config_bundle()`. Bundle propagation uses W3C baggage headers, and each version links to its parent via `parentVersionIds`, giving a full lineage chain. Because a bundle update takes effect on the *next invocation with no redeploy*, this is functionally a feature flag for anything the agent’s own logic branches on — a team can encode “if `config.enable_new_tool_selection_logic`, use path A, else path B” directly in agent code and flip it by publishing a new bundle version, with the same rollback-by-repointing-a-version property Runtime/Harness endpoints have (Part 3.4).

Third-party, purpose-built: LaunchDarkly AgentControl. LaunchDarkly is available as an AWS Marketplace integration specifically scoped to Bedrock/AgentCore (“AgentControl,” alongside a separate “CodeControl” product for AI-generated code). It adds capabilities Configuration Bundles alone do not provide: percentage-based progressive rollouts with fine-grained targeting rules (individual users, contextual segments, prerequisite flags), **runtime behavior control without redeployment** (prompt/model configuration changes applied live), and — notably — an **automatic-corrective-action** capability that can revert a flag without human intervention when monitored impact metrics cross a defined threshold. A documented pattern (the AWS DevOps Agent + LaunchDarkly integration, part of the June 2026 summit wave covered in Part XVII) uses feature flags as an **incident-containment mechanism**: when a deployment is classified high-risk and lacks flag coverage over the changed code path, the DevOps Agent recommends wrapping it in a flag before shipping; during an active incident, the same agent can correlate error spikes against recent flag changes and recommend disabling the implicated flag — which is frequently a faster containment action than a full rollback — as an explicit alternative to the kill-switch and circuit-breaker mechanisms in Parts 45–46.

When to reach for which: use Configuration Bundles when the gate is purely AgentCore-native configuration (prompt, model, tool description) and the audience is “some fraction of sessions via A/B test.” Use a dedicated feature-flag platform (LaunchDarkly or equivalent) when you need fine-grained audience targeting beyond simple percentage splits (specific customers, specific regions, staged internal-then-external rollout), when the flag needs to gate application-level behavior outside AgentCore entirely (e.g., a flag shared between the agent and a non-agent service), or when automatic threshold-triggered rollback without a human in the loop is a requirement.

24.6 48. Resume Workflow

Three distinct “resume” scenarios exist on AgentCore, each with its own documented mechanism:

Resume a conversation across invocations (session continuity). Supplying the same `runtimeSessionId` on subsequent calls routes to the same microVM (best-effort, per Part III.3.2) and preserves in-memory state; for durability beyond a single microVM’s lifetime, AgentCore Memory’s short-term store persists session events independently of the compute

layer, so a conversation can resume correctly even after the underlying microVM has been recycled and replaced — the Harness documentation is explicit that short- and long-term memories persist “across sessions, even when the underlying microVM session has expired and is replaced by a new one.”

Resume a long-running background task within a session. The `add_async_task/complete_async_task` SDK pair, combined with the `/ping` health contract, is the documented pattern: a task starts, is tracked by ID, runs on a background thread while the ping handler reports `HealthyBusy` (preventing the platform from treating the session as idle or hung), and completion is signaled explicitly. Critically, **the entrypoint handler must never block**, because a blocked entrypoint also blocks the `/ping` responder on a single-threaded implementation — the documented failure mode is indistinguishable from a genuine hang from the platform’s point of view, and AWS’s troubleshooting guide calls this out explicitly as a common misconfiguration. A separate, well-documented gotcha: the default entrypoint request timeout is **60 seconds**; work exceeding that without a ping/async-task signal is treated as a failure and retried per Runtime’s delivery policy, which is the root cause behind a commonly-reported “AgentCore keeps calling my entrypoint every 60 seconds” support pattern. The correct fix is either the async-task/ping pattern above, or offloading to Step Functions/SQS and returning an immediate acknowledgment.

Resume after a human-in-the-loop pause spanning hours or days. This is a Step Functions concern, not a Runtime concern: the `.waitForTaskToken` callback pattern pauses a state-machine execution indefinitely (Step Functions executions can run up to one year), holding a task token that an external approval application later returns via `SendTaskSuccess/SendTaskFailure` to resume exactly where execution left off. `TimeoutSeconds/HeartbeatSeconds` on the waiting state, paired with a `Catch` clause routing to an escalation state, is the documented safe-fallback pattern — **AWS’s own Well-Architected Agent AI Lens guidance is explicit that an approval state without a timeout will run (and be billed) indefinitely if the approval is simply forgotten**. For agents built on Harness specifically, the June 2026 Step Functions integration lets a harness-powered reasoning step sit directly inside such a workflow, wrapped by the same human-approval and error-handling states used for any other Step Functions task.

24.7 49. Failover and Timeout/Retry Discipline

Beyond the multi-region DR patterns in Part XIV, AgentCore’s request-level failover behavior has two documented, non-obvious characteristics worth designing around explicitly:

- **Default request timeout is 60 seconds at the entrypoint**, independent of any session-level `maxLifetime/idleRuntimeSessionTimeout`. Anything long-running must use the `async-task/ping` pattern (Part 48) or be offloaded entirely; naive long-running synchronous handlers will be retried by the platform’s own delivery policy, which — absent idempotency in the handler — can produce duplicate side effects (duplicate tool calls, duplicate emails, duplicate charges) on retry. **Idempotency keys on every state-mutating tool call are a direct mitigation for this specific failover behavior**, not merely general good practice.
- **Gateway exposes granular, per-error-class CloudWatch metrics** — `Invocations`, `Throttles [429]`, `SystemErrors [5xx]`, `UserErrors [4xx]` (4xx excluding 429), `Latency`, `Duration`, and `TargetExecutionTime` — which is the correct signal set to drive automated failover/circuit-breaking decisions rather than a single aggregate error rate: a spike in `Throttles` calls for backoff, a spike in `SystemErrors` calls for a downstream-target health check, and a spike in `UserErrors` calls for a client-side/authorization investigation — three different runbooks behind what a naive dashboard might show as one undifferentiated “errors” line.
- **Recommended retry policy, consistent with general distributed-systems prac-**

tice and specifically validated against Gateway’s error taxonomy: retry 429 and 5xx with exponential backoff and jitter; do **not** retry 4xx (other than 429) without first modifying the request — a 403 from a Cedar forbid or an expired token will not succeed on blind retry and will instead amplify load during exactly the kind of incident (a bad policy push, an expired credential) where amplification is most damaging.

24.8 50. Exception Handling at the Agent, MCP, and Tool Layer: Auth vs. Business Exceptions

A production-ready agent needs to treat these as **structurally different exception classes**, because the correct response differs completely:

Exception class	Example	Where it surfaces	Correct handling
Infrastructure / transient	5xx from a tool target, network timeout, Bedrock ThrottlingException	Gateway SystemErrors/Throttles metrics; Bedrock invocation logs	Retry with exponential backoff + jitter; circuit-break after N consecutive failures (Part 46)
Authentication / authorization	Expired OAuth token, invalid SigV4 signature, missing IAM permission, Cedar forbid (403)	Gateway UserErrors [4xx]; AccessDeniedException in CloudTrail/logs	Never blind-retry. Refresh via Token Vault if the credential is merely expired; if it’s a Cedar denial, this is authorization working as designed — surface it to the agent as “not permitted,” not as a bug, and do not attempt to route around it
Business-rule / policy violation	Refund amount exceeds Cedar-enforced limit, geography-restricted action, insufficient approval	Cedar forbid at the Gateway (same 403 surface as auth denials, but semantically different: this is <i>deliberate governance</i> , not a system failure)	Feed the denial reason back to the agent’s reasoning so it can inform the user or seek human approval (Part 51) rather than silently failing; this is precisely why AgentCore Policy evaluates <i>before</i> the tool executes — the denial is a first-class outcome, not an exception path

Exception class	Example	Where it surfaces	Correct handling
Input / validation	Malformed tool arguments, schema mismatch	Tool/Lambda-level exception, surfaced back into the agent's context as a ToolResult error	Strands' default behavior (Part XI.21) is to let this bubble up and terminate the agent unless the developer wraps it — production code must add an explicit error boundary here that returns a structured error back into the model's context so it can retry with corrected arguments, rather than crashing the session
Silent / no-error-signal failure	Wrong tool selected, subtly incorrect output, degraded personalization	No 4xx/5xx at all — this is the class Optimization's failure-insights (Part 26) exists specifically to catch	Continuous online evaluation, not exception handling — this class cannot be caught by any try/except because nothing "throws"

The practical architectural point: **Cedar-mediated business-rule denials and true authentication failures share the same HTTP-level surface (403) but require opposite handling** — one should never be retried and should be treated as a system integrity signal (possible credential compromise, possible attack), the other is expected, routine, and should be designed into the agent's own conversational flow (e.g., "I can't process a refund over \$500 without manager approval — would you like me to request approval?"). Conflating the two in a single generic except Exception handler is a documented anti-pattern this report adds explicitly alongside those in Part XIX.

24.9 51. Human-in-the-Loop Switch

AWS's own healthcare/life-sciences reference architecture (April 2026) documents four distinct, composable HITL patterns, each fitting a different latency/audit/control-ownership profile:

1. **Hook-based blanket policy** — a Strands HookProvider registered on BeforeToolCallEvent intercepts *every* tool call before execution and enforces a centralized approval policy without modifying individual tools. Best for a uniform rule ("everything tagged sensitive pauses"), enforced once at the agent-loop level.
2. **Tool-context, per-tool RBAC** — approval logic embedded inside each tool definition itself, allowing fine-grained, tool-specific rules (e.g., role-based: a nurse can view vitals without approval, a discharge order requires physician sign-off). More flexible than the hook pattern but requires touching every sensitive tool individually.
3. **Step Functions remote interrupt (asynchronous, third-party approver)** — the agent session is not blocked; a Step Functions execution sends a notification (SNS/email/Slack/Teams) to an external approver, the agent returns an immediate acknowledgment to the end user, and the workflow resumes via waitTaskToken once

the approver responds, potentially hours or days later (Part 48). This is the correct pattern whenever the approver is a *different person* from the end user, or when approval may take longer than any reasonable synchronous wait.

4. **MCP Elicitation (`ctx.elicit()`), real-time, protocol-native** — the MCP server itself requests approval mid-tool-call via the elicitation protocol; on AgentCore Runtime this is relayed to the end user over a WebSocket connection in real time. This keeps the approval logic entirely inside the MCP server’s tool definitions — **the agent itself has no knowledge of which tools require approval**, so approval requirements can be added or changed independently of the agent’s own code, a meaningful decoupling property for organizations where the tool owner and the agent owner are different teams.

Cross-cutting guidance from AWS’s Well-Architected Agentic AI Lens (AGENTSEC04-BP02): risk-tier the decision, don’t route everything through a human (“routing every agent action through human review produces rubber-stamp approvals; routing none produces unbounded autonomy”) — classify actions and match the approval mechanism to the execution environment (Step Functions callbacks for step-function-driven agents; AgentCore Runtime async tasks, per Part 48, for long-running approval processing embedded in a conversational session). Every approval mechanism needs an explicit timeout with a **safe-fallback default that blocks the operation**, not one that defaults to allow, and every decision — reviewer identity, notification/response timestamps, the operation under review, and any escalation events — needs to be logged durably for audit, independent of whichever transport (SNS, MCP elicitation, hook) carried the approval request.

24.10 52. Sampling

“Sampling” means two unrelated things in this stack, and both matter operationally:

MCP protocol sampling (`sampling/createMessage`) is a first-class MCP capability, distinct from tools/resources/prompts, that lets an **MCP server** request an LLM completion **from the client** — reversing the normal request direction. A code-review MCP server, for instance, can ask the client’s LLM to summarize a diff mid-tool-call without the server holding its own model credentials; the client retains full control over model selection and access, and the protocol’s human-in-the-loop design gives the user two explicit checkpoints — approve/edit the outbound prompt before it reaches the LLM, and approve/edit the completion before it’s returned to the server. A draft protocol revision under discussion as of mid-2026 (referenced in spec-tracking commentary as version-dated “2026-07-28”) proposes an `InputRequiredResult` mechanism that would let a `tools/call` pause mid-flight for either an elicitation or a sampling request and resume later by re-issuing the call with the gathered answers plus an opaque, client-echoed `requestState` — explicitly designed so a stateless server instance can resume the interaction without holding a connection open, which is directly relevant to the Runtime statelessness discipline covered in Part X. Treat this as an emerging capability to watch, not a currently-guaranteed one.

Observability trace sampling is the unrelated, cost-control meaning: capturing telemetry for only a subset of requests to bound instrumentation overhead and CloudWatch/X-Ray ingestion cost at scale. Documented guidance converges on an environment-dependent default: **100% trace sampling in development** (full visibility while iterating), stepping down to **adaptive sampling in production** — CloudWatch Application Signals’ adaptive sampling, or a fixed ratio such as 1-in-10 with 100%-on-error as a common independently-recommended pattern, so that every *failed* request is still fully captured even when most successful ones are not. Two AWS-specific mechanics worth noting: **X-Ray Trace Indexing only indexes 1% of spans as searchable trace summaries by default** even when full spans are ingested (a distinct dial from raw ingestion sampling, and one teams commonly discover only after failing to find a specific trace in the console), and **CloudWatch Transaction Search must be explicitly enabled once per account per Region** before AgentCore spans appear in the

GenAI Observability dashboard at all — an easy-to-miss one-time setup step, not an ongoing sampling decision.

24.11 53. Synthesis: The Full “Keep It Live” Stack

Read top-to-bottom, these mechanisms form a layered resilience stack, each layer catching what the one above it misses:

1. **Governance (pre-execution):** Cedar Policy in ENFORCE mode — blocks disallowed actions before they ever reach a tool.
2. **Circuit breaking (mid-execution, automated):** Lambda-interceptor-fed counters + Cedar budget rules, or a third-party overlay — trips on threshold breach without waiting for a human.
3. **Kill switch (mid-execution, manual/emergency):** StopRuntimeSession for a single session; a pre-staged IAM emergency-deny for a stuck session; your own session registry for fleet-wide visibility AgentCore does not natively provide.
4. **Human-in-the-loop (mid-execution, judgment-requiring):** hook, tool-context, Step Functions, or MCP elicitation — matched to latency and ownership requirements per Part 51.
5. **Exception discipline (per-call):** the five-way taxonomy in Part 50, so retry logic, credential refresh, and business-rule denials never get routed through the same generic handler.
6. **Progressive rollout (change management):** configuration-bundle or target-based A/B testing with statistical-significance gating before every promotion.
7. **Load/regression testing (pre-change validation):** batch Evaluations in CI/CD, plus concurrency-focused load testing against known quota ceilings, before a change ever reaches the A/B stage.
8. **Observability (continuous, underlying everything above):** OTEL traces feeding both CloudWatch and a secondary platform, sampled appropriately for environment and cost, with failure-insights continuously mining for the silent failures no exception handler will ever catch.

No single AgentCore service in isolation delivers “keep the agent live” — it is the composition of Policy, Gateway interceptors, Runtime’s session/ping/async-task primitives, Optimization’s A/B testing, Step Functions’ callback pattern, and a small amount of platform-team-owned glue (a session registry, a pre-staged emergency-deny policy, a circuit-breaker Lambda) that AgentCore does not yet provide natively. Architects should budget for that glue explicitly rather than assuming it ships with the platform.

25 Appendix A — Sources

This report is grounded in the following categories of publicly available evidence gathered via web search in July 2026: AWS official documentation (docs.aws.amazon.com/bedrock-agentcore), AWS Machine Learning, Security, Networking, and AWS News blogs, AWS re:Invent 2025 and AWS Summit New York (June 17, 2026) announcements, the *strands-agents* GitHub organization and its published SDK documentation (strandsagents.com), independent security research from Palo Alto Networks Unit 42, BeyondTrust Phantom Labs, and Sonrai Security, and practitioner/analyst write-ups from AWS Heroes, AWS Builder Center, Arize AI, Weights & Biases, DeepWiki ([aws-labs/amazon-bedrock-agentcore-samples](https://aws-labs.github.io/amazon-bedrock-agentcore-samples)), and multiple independent cloud-architecture blogs (hidekazu-konishi.com, clawaws.com, [cloudvisor](https://cloudvisor.com), [xebia](https://xebia.com), dev.to contributors) current as of late June-early July 2026. Facts drawn from AWS marketing/keynote claims not yet independently verified are flagged explicitly in-text as such (notably in Part XVI and Part XVII); all other claims are drawn from official documentation, release notes, or independently reproduced/verified research.

26 Appendix B — Glossary

2LO / 3LO — Two-legged / three-legged OAuth: machine-to-machine client-credentials grant vs. user-delegated authorization-code grant. **A2A** — Agent-to-Agent protocol, for direct agent-to-agent communication and discovery. **Cedar** — AWS’s open-source, CNCF-hosted authorization policy language, used by both Amazon Verified Permissions and AgentCore Policy. **Firecracker** — The open-source microVM virtual machine monitor underlying Runtime, Browser, Code Interpreter, Lambda, and Fargate isolation. **MCP** — Model Context Protocol, the open standard for connecting agents to external tools and data sources. **OBO** — On-Behalf-Of token exchange; Gateway exchanges an inbound user token for a downstream-scoped token carrying both user and agent identity. **OTEL / ADOT** — OpenTelemetry / AWS Distro for OpenTelemetry, the instrumentation standard underlying AgentCore Observability. **Workload Identity** — An agent/Runtime’s own first-class security identity, distinct from and combined with, but never substituting for, the end user’s identity.